A Survey of FPGA-based 3D CNN Accelerators and Hardware-aware Algorithmic Optimizations

DANA DIACONU, Electrical and Computer Engineering, Northeastern University, Boston, United States
XUE LIN, Electrical and Computer Engineering, Northeastern University, Boston, United States
MICHAELA BLOTT, Advanced Micro Devices Inc, Dublin, Ireland
MIRIAM LEESER, Electrical and Computer Engineering, Northeastern University, Boston, United States

3D Convolutional Neural Networks (3D CNNs) can outperform 2D CNNs on several tasks, including action recognition, video captioning, abnormal event detection, and medical image interpretation. Compared to 2D CNNs, 3D CNNs have a larger number of parameters and higher computational complexity. For this reason, researchers have focused on designing efficient 3D CNN architectures and hardware accelerators. The purpose of this survey is to serve as a guide to recent work on 3D CNNs for action recognition with a focus on FPGA-based accelerators and hardware-aware algorithmic optimizations. We provide an overview of the state-of-the-art in 3D CNN architectures as well as the action recognition datasets used for training and testing the architectures. A performance comparison of 2D versus 3D CNNs on two datasets (Sports-1M and UCF101) is included. We explore the designs of FPGA-based accelerators and compare them in terms of achieved throughput, resource utilization, and power. We survey current methods of optimization for 3D CNN architectures, which are meant to reduce the number of parameters and the memory requirements and to facilitate their deployment on FPGAs. Finally, we highlight current challenges and potential areas of improvement in acceleration of 3D CNNs on FPGA platforms.

CCS Concepts: • **Hardware → Hardware accelerators**; • **General and reference → Surveys and overviews**; • **Computing methodologies → Neural networks**; **Machine learning**; **Computer vision**; **Activity recognition and understanding;**

Additional Key Words and Phrases: 3D CNN, FPGA, Hardware Accelerators, Action Recognition

ACM Reference Format:
Dana Diaconu, Xue Lin, Michaela Blott, and Miriam Leeser. 2025. A Survey of FPGA-based 3D CNN Accelerators and Hardware-aware Algorithmic Optimizations. *ACM Comput. Surv.* 58, 6, Article 155 (December 2025), 35 pages. https://doi.org/10.1145/3777366

1 Introduction

Three Dimensional Convolutional Neural Networks (**3D CNNs**) have been shown to outperform **two Dimensional Convolutional Neural Networks** (**2D CNNs**) on a number of tasks, due

This material is based upon work supported by the National Science Foundation (NSF) under Grant CCF-1901378.
Authors' Contact Information: Dana Diaconu (corresponding author), Electrical and Computer Engineering, Northeastern University, Boston, Massachusetts, United States; e-mail: diaconu.d@northeastern.edu; Xue Lin, Electrical and Computer Engineering, Northeastern University, Boston, Massachusetts, United States; e-mail: xue.lin@northeastern.edu; Michaela Blott, Advanced Micro Devices Inc, Dublin, Ireland; e-mail: michaela.blott@amd.com; Miriam Leeser, Electrical and Computer Engineering, Northeastern University, Boston, Massachusetts, United States; e-mail: m.leeser@northeastern.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0360-0300/2025/12-ART155
https://doi.org/10.1145/3777366

to the fact that 3D CNNs handle temporal information as well as the spatial data used by 2D CNNs. When we apply a 2D CNN to a video, the network processes the video frame by frame and gives a prediction for each image, which can make the final classification fluctuate wildly. One solution to fast changes in the results is to average the predictions over a sequence of frames; however, this can still lead to wrong predictions because the network cannot learn from context and it does not take into account the order of the frame sequence. Take as an example the opening or closing of a door. If we analyze the frame in which a person holds the handle and the door is half open, we cannot predict if that person is opening or closing the door unless we have information from the previous and/or the following frames in the original order. We need to make use of temporal information for a correct classification. 3D CNNs show great potential in applications such as Human Action Recognition [31], which is useful for autonomous driving, video captioning [8] and medical image interpretation [55]. They have also been applied to motion prediction [45], abnormal event detection [76] (which is useful for intelligent video surveillance), and hand gesture detection [2].

This survey has two main goals. The first is to serve as a guide to recent work on the implementation of 3D CNNs with a focus on FPGA-based accelerators and hardware-aware algorithmic optimizations. We focus on the FPGA-based acceleration of 3D CNNs for action recognition. Action recognition is a common video classification subtask among event detection [32], anomaly detection [49], gesture recognition [2] and many more. The second goal is to highlight current challenges and potential areas of improvement in the acceleration of 3D CNNs on FPGA platforms.

3D CNNs on FPGAs represent a good tradeoff between accuracy versus number of parameters and number of FLOPS which makes them suitable for being deployed on devices with limited memory and in applications which require high throughput and low latency. FPGAs have been shown to deliver good performance with low power consumption compared to other architectures as illustrated in Section 5.

To provide a general idea of how 3D CNNs compare to different architectures in terms of Video@Hit1 (or Top-1) accuracy, FLOPS (number of floating point operations (multiply-adds)) and number of parameters, we present the Pareto fronts for the Kinetics dataset [34] (action recognition dataset described in Section 3). Figure 1 shows the Pareto front for number of FLOPS. Due to the fact that many articles do not include this metric but many more provide the number of parameters, we also show the Pareto front comparing accuracy with number of parameters for the same architectures in Figure 2. Both graphs show that vision transformers dominate the state-of-the-art on action recognition (particularly on the Kinetics dataset in this case), meaning they achieve the highest accuracies, but at a cost of a high number of FLOPS and parameters.

Figure 3 shows the high-level overview of designing hardware accelerators for 3D CNNs and motivates the organization of the article. Hardware-aware algorithmic optimizations are applied to the 3D CNN such that it can be implemented in hardware with as few resources as possible while maintaining accuracy close to the original (before optimizations). This will facilitate reaching high throughput and low latency, the main hardware performance metrics of accelerators for machine learning. Most of the optimizations we present are useful even when not targeting hardware implementations; however, in the context of this article, we refer to them as hardware-aware optimizations because the end goal is to design efficient accelerators. It is also useful to have an overview of the datasets which are used for training the 3D CNNs for the action recognition task and to identify the advantages and disadvantages of each training dataset. Section 3 presents a comparison between current open-source action recognition datasets.

We aim at answering the following questions: “Which are the datasets currently available?”, “What and how many categories do they include?”, “What is the size (number of videos) of each dataset?” and “To what extent are these datasets used in the research community?”. Then, we are

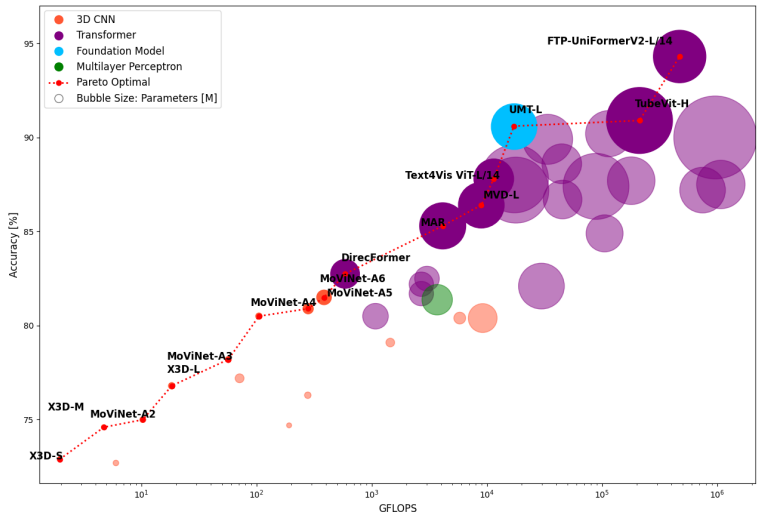


Fig. 1. Comparison between 3D CNNs, Vision Transformers, Foundation models and other architectures in terms of accuracy, GFLOPS and number of parameters (represented by the size of the bubbles) on the Kinetics dataset with Pareto optimal points identified.

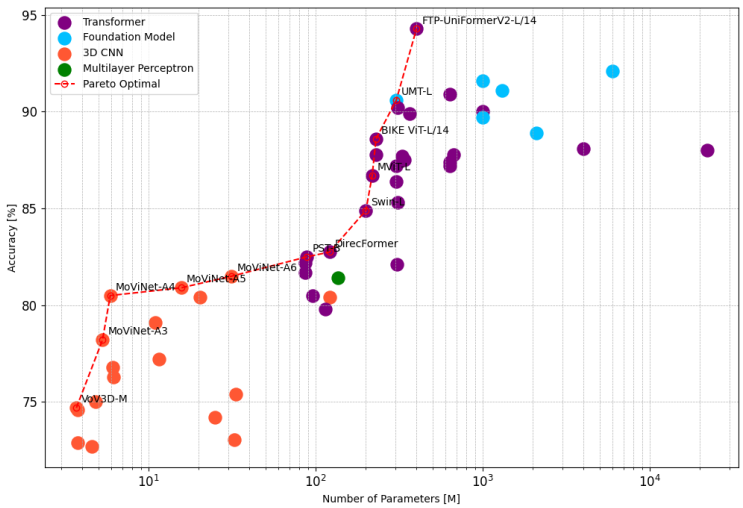


Fig. 2. Comparison between 3D CNNs, Vision Transformers, Foundation models and other architectures in terms of accuracy and number of parameters on the Kinetics dataset with Pareto optimal points identified.

interested in the architectures of the 3D CNNs: how do they differ, what is their size in terms of number of parameters and how do they perform on these different datasets? Section 2 goes over the main background specifics of 2D and 3D CNNs, Section 4 compares the performance of 2D and 3D CNNs on two action recognition datasets and Section 6 focuses on the state-of-the-art architectures of 3D CNNs. Section 5 compares FPGA 3D CNN accelerators with CPUs and GPUs. Section 7 focuses

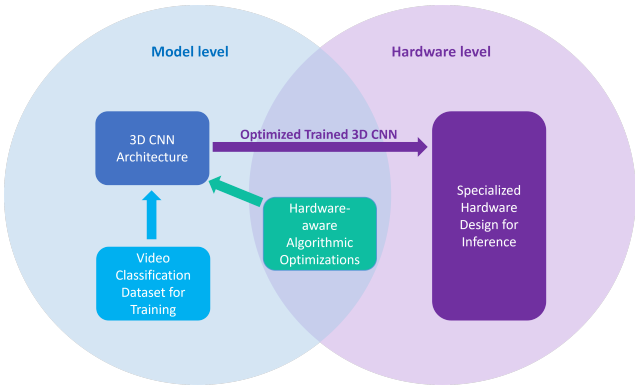


Fig. 3. High-level overview of designing hardware accelerators for 3D CNNs.

on the FPGA accelerators for 3D CNNs, and the state-of-the-art in terms of design architectures and techniques. The questions that we address in Section 8 are: "What kind of techniques are used to optimize the 3D CNNs for hardware implementation?", "How do we reduce the number of parameters but maintain the accuracy?", "How do we simplify the computations?", and "What effect do these optimizations have on the performance (accuracy, throughput, and latency) of the hardware implementation?". Finally, Section 9 discusses the research gaps and highlights potential areas of improvement.

The main contributions of this article are:

— Overview and comparison of current action recognition datasets.
— Overview of state-of-the-art architectures for 3D CNNs and comparison in terms of model size and accuracy achieved on various action recognition datasets.
— Exploration of state-of-the-art FPGA-based accelerators for 3D CNNs and comparison in terms of optimization methods, throughput, resource utilization and power consumption.
— Overview of hardware-aware algorithmic optimizations for 3D CNNs such as pruning and quantization.

There are a number of applications of 3D CNNs not considered here. For the interested reader, Ren et al. [48] cover the state-of-the-art of machine learning architectures for 3D skeleton-based action recognition and Niyas et al. [44] survey medical image segmentation with 3D CNNs. Note that these surveys do not talk about hardware acceleration.

2 Background

Convolutional Neural Networks (**CNNs**) are built with three main types of layers: convolutional, pooling, and fully-connected layers. The convolutional layer extracts features from the input images and outputs a feature map which is fed to the next layers. The pooling layer is used to decrease the dimensions of the feature maps in order to reduce the computation and number of weights needed. The most common types of pooling layers are Max Pooling and Average Pooling where the maximum and the average values, respectively, are deduced from the input feature map. The fully-connected layer is usually placed before the output layer of the network and its neurons are connected to all neurons obtained from flattening the previous feature map. Convolutional layers and fully-connected layers are typically followed by an activation function which adds non-linearity and helps the network decide which neurons need to be activated, meaning which neurons will contribute to the output. The most commonly used types of activation functions are the **Rectified Linear Unit** (**ReLU**) and Sigmoid functions.

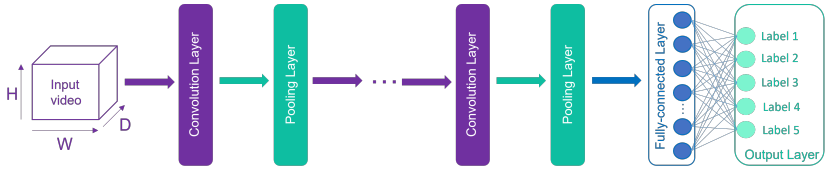


Fig. 4. 3D CNN base architecture.

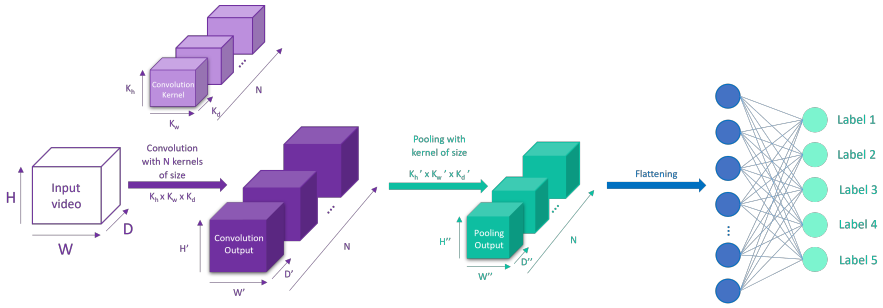


Fig. 5. 3D CNN base architecture detailed.

Figure 4 shows a general diagram of a 3D CNN where the input is a video of size H x W x D (data shapes are described in detail in the next paragraph) and the output consists of the top 5 classifications. Figure 5 shows the same base 3D CNN architecture, with one convolution layer and one pooling layer, in more detail.

2D CNNs process spatial data, whereas 3D CNNs process spatial plus temporal data. Figure 6 is a general representation of an input for a 3D CNN. H and W represent the height and width of the image, C represents the number of channels, and N represents the number of samples. All these dimensions (H, W, C, and N) are valid for 2D CNNs, where the number of samples N means the number of images, but for 3D CNNs there is an extra dimension, D, which is the depth of the video, i.e., the number of frames while N is the number of videos. High-dimensional data is represented through tensors; 2D CNNs use 4D tensors (or 3D if N=1), while 3D CNNs use 5D tensors (or 4D if N=1), where the extra dimension is the temporal axis.

$$\text{Output}(x,y,z) = \sum_{i=0}^{K_d-1}\sum_{j=0}^{K_h-1}\sum_{k=0}^{K_w-1} \text{Input}(x+i,y+j,z+k)\cdot\text{Kernel}(i,j,k), \tag{1}$$

$$\text{Output}(x,y,z) = \max_{i=0}^{K_d-1}\max_{j=0}^{K_h-1}\max_{k=0}^{K_w-1} \text{Input}(x+i,y+j,z+k). \tag{2}$$

Table 1 shows the general computational complexity of 1D, 2D, and 3D convolutions. 3D CNNs come with a larger number of parameters and higher computational complexity than 2D CNNs which makes the design of specialized hardware accelerators more challenging. The general mathematical formulations for 3D convolution and 3D pooling operations are shown in Equation 1 where $K_h, K_w,$ *and* K_d represent the dimensions of the convolution kernel and Equation 2 where $K_h, K_w,$ *and* K_d represent the dimensions of the pooling kernel, respectively.

3 Action Recognition Datasets

This section provides an overview of the datasets that are commonly used for training 3D CNNs and compares them in terms of number of categories, types of classes and number of videos. Each

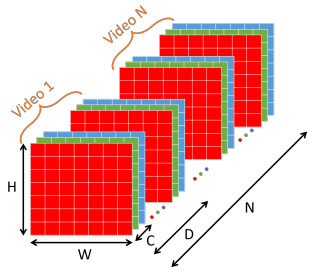


Fig. 6. General input representation for a 3D CNN. H and W represent the height and width of the image, C represents the number of channels, D is the depth of the video/number of frames and N represents the number of samples/videos.

Table 1. Computational Complexity of 1D, 2D, and 3D Convolutions

Conv type	Input size	Kernel size	Output size	Computational Complexity [FLOPS]
1D Conv	L	K	O = ((L-K +2*P)/S + 1)*N	$2 * O * K * N$
2D Conv	H x W	$K_h \, x \, K_w$	$O_h = (H - K_h + 2P)/S + 1$ $O_w = (W - K_w + 2P)/S + 1$ $O_c = N$	$2 * O_h * O_w * K_h * K_w * N$
3D Conv	H x W x D	$K_h \, x \, K_w \, x \, K_d$	$O_h = (H - K_h + 2P)/S + 1$ $O_w = (W - K_w + 2P)/S + 1$ $O_d = (D - K_d + 2P)/S + 1$ $O_c = N$	$2 * O_h * O_w * O_d * K_h * K_w * K_d * N$

L represents the length of the input sequence for 1D convolution. H represents the height and W represents the width for the 2D and 3D convolutions (spatial dimensions). D (temporal dimension) stands for depth in 3D convolution. We assume a stride S of size 1 and padding P of size 0 and one input channel. N is the number of samples and O is the output size.

action recognition dataset is introduced and we present their advantages and disadvantages. The performance and generalization capability of CNNs to unseen data is greatly impacted by the training dataset. Table 2 summarizes the most recent and most used action recognition datasets.

Previous action recognition datasets, such as the KTH dataset [52] (2004) and the Weizmann dataset [21] (2007), are based on a small number of subjects who were recorded doing several actions in a controlled environment. The algorithms trained on this kind of dataset have a limited generalization capability and low performance in real applications where the variety of subjects, backgrounds, lighting and camera positions can be overwhelming.

HMDB51 [39] was built from various sources, including movies and YouTube, and all videos were manually annotated. Besides annotating the videos with the class of action it belongs to, the authors and their team added additional information such as the video quality (high, medium, low), camera movement (moving or static) and number of people involved (one, two, or multiple). This kind of information is useful for comparing the performance and finding the limitations of action classification algorithms on subcategories of videos, such as low quality versus high quality videos. The HMDB51 dataset is relatively old and limited in size, having only 51 action classes, which is why it has become less prominent in current research.

UCF101 [56] consists of 101 action categories with 25 groups per action. One group contains 5-7 clips that share common features (e.g., background) and all videos come from YouTube. UCF101 is a challenging dataset for action recognition algorithms because it includes shaky footage, videos shot in poor lighting and backgrounds which make it difficult to discern the subjects, with shadows or over exposure. However, algorithms that perform well on challenging datasets are more suitable for

Table 2. Action Recognition Datasets

Name, Release Year	Size	Types of classes
HMDB51 [39], 2011	51 action classes Around 101 videos per class Around 7k videos Video length: min 1 second	General facial actions (e.g., laugh, chew) Facial actions with object manipulation (e.g., eat, drink) General body movements (e.g., clap, sit up) Body movements with object interaction (e.g., brush hair) Body movements for human interaction (e.g., hug)
UCF101 [56], 2012	101 action classes Around 125-175 videos per class Around 13k videos Video length: 1-71 seconds	Human-Object Interaction Body-Motion Only Human-Human Interaction Playing Musical Instruments Sports
Something-Something V2 [22], 2017	174 action classes Around 620 videos per class Around 108k videos Video length: 2-6 seconds	Human-Object Interaction such as: Holding [something], Spilling [something] behind [something], Moving [something] away from the camera, and so on
ActivityNet [27], 2015	203 action classes Around 137 videos per class Around 28k videos Video length: 300-600 seconds	Personal care, Household Eating and Drinking, Working Caring and Helping, Sports Socializing and Leisure
Kinetics [34], 2017	400 action classes Around 400 videos per class Around 160k videos Video length: 10 seconds	Person Actions (e.g., drawing, laughing) Person-Person Actions (e.g., hugging, shaking hands) Person-Object Actions (e.g., opening presents)
Sports-1M [33], 2014	487 action classes Around 1000-3000 videos per class Around 1M videos Video length: 336 seconds	Sports such as: Aquatic Sports, Team Sports, Winter Sports, Ball Sports, Combat Sports, Sports with Animals
YouTube-8M [1], 2016	4800 action classes Different number of videos per class Around 8M videos Video length: 230 seconds	Part of the top categories: Arts and Entertainment Sports, Science, News, Animals Games, Hobbies, Travel, Shopping

integration in real applications, such as autonomous driving. Although UCF101 has been around for a while and is relatively small compared to more recent datasets, it remains a key reference point for comparing the performance of 3D CNNs in research. Its size also allows for quicker training times than larger datasets, making it easier to evaluate models.

The Something-Something V2 [22] dataset focuses on Human-Object Interaction but, compared to other datasets which contain this action type, such as UCF101, the labels are more detailed about the physical aspects of the scenes. The goal is for the model to better understand the actions and to generalize better to interactions with multiple objects. The class labels use the "something" placeholder for the objects and they are called caption-templates. An example of a detailed class label, which is not common in any other dataset, is "Pulling two ends of 'something' so that it separates into two pieces". All videos come from crowd workers who act out the templates with objects of their choice. A model trained on this dataset should be able to generalize better and recognize that the same action is used with several different objects. For instance, instead of only learning the action of opening the door, it could correctly identify the action of opening "something" where "something" could be the bag, the zipper, the blinds or the mouth. Something-Something V2 has 174 action classes, but compared to UCF101 it has around 4x more videos per class.

ActivityNet [27] is a dataset with 203 action classes which contain a wide range of activity types. It is organized as an activity hierarchy with multiple levels of depth for each category (e.g., Household activities -> Lawn, garden -> Plant Care -> Watering garden). The videos in ActivityNet were collected from multiple online sources and some of them have been kept untrimmed, while others have been trimmed to a specific activity. Usually, the untrimmed videos involve mixed activities, so they have been annotated with multiple classes. However, untrimmed videos can contain unrelated frames which substantially complicate the training of the action recognition

networks. Even though ActivityNet has more classes than Something-Something V2, it has almost 4x fewer videos.

Kinetics [34] contains 4x and 8x more action classes than UCF101 and HMDB51 datasets, respectively. The authors report that compared to UCF101, which has clips from only 2.5k distinct videos, each clip in Kinetics is extracted from a different Youtube video. A lot more variety is provided by not having clips with high similarity in terms of the same subjects, backgrounds and so on. The original size of the dataset was 400 classes, but since then the dataset has been extended to 600 [5] and to 700 classes [6]. Throughout the article we use the following naming convention: Kinetics for the original dataset with 400 classes and Kinetics-600/700 for the extended dataset. Similarly to UCF101, Kinetics is a widespread reference point used in comparing different models.

Sports-1M [33] contains YouTube videos as well, but it is larger than UCF101, with 1 million videos as the name suggests, and it comprises only sports classes. The videos were annotated using their metadata, but the correctness of the metadata is not ensured. The authors state that approximately 5% of the videos were annotated with multiple classes and that there could be a lot of variation between the frames if, for example, the scoreboard or the crowd were also filmed. Furthermore, there may be duplicate videos in the dataset which means there is a possibility to have information leaked from the training set to the test set. However, their analysis of the dataset shows that the chance of this kind of information leakage, especially if sets of short clips are used, is slim. They report that only 1,755 out of 1 million videos contain a considerable number of near-duplicate frames.

YouTube-8M [1] is a dataset with multiple machine-generated labels based on various data such as metadata or query click signals. The labels are filtered using automated and manual methods, and a deep CNN – the Inception network trained on ImageNet – is used to extract frame level features. This dataset contains 8 million videos and features for 1.9 billion video frames. In comparison to Sports-1M and ActivityNet, YouTube-8M contains not only activities, but also objects, scenes, and events. Abu-El-Haija et al. [1] test if the models trained on YouTube-8M can generalize to other datasets. They evaluate transfer learning on Sports-1M and ActivityNet datasets and show that pre-training on YouTube-8M is valuable.

4 State-of-the-Art Performance of 2D and 3D CNNs on the Action Recognition Task

The first question that comes to mind when thinking about neural networks for video processing is: How efficient are 3D CNNs compared to 2D CNNs and is the increase in the number of parameters and the computational complexity worth it? This section compares the performance of 2D and 3D CNNs on action recognition datasets.

Table 3 and Table 4 show the accuracy of both state-of-the-art 2D and 3D CNNs on the Sports-1M and UCF101 datasets. Hit@1 and Hit@5 represent the accuracy of the 3D CNNs where the classification is the same as the target label and where the target label is one of the top 5 predictions, respectively. The performance of the networks on UCF101 is expressed as the 3-fold accuracy which is the mean Video Hit@1 accuracy obtained on 3 different train/test splits of the dataset. The 3 train/test splits are defined by the authors of the dataset and are provided on the official website. These splits can be used to test the generalization capability of the network on different subsets of the dataset. It is common for results to be reported as the 3-fold accuracy when testing the network on all 3 splits and averaging the results.

Table 4 clearly shows the advantage of 3D CNNS. As seen in Table 3, the results are more mixed. The main advantage of 3D CNNs over 2D CNNs is that they process the temporal dimension along with the spatial dimensions and they can capture and understand motion. Especially with regards to the Sports-1M results, researchers have found ways to help 2D CNNs take into account the temporal dimension as well. Note that all the results presented for the Sports-1M dataset use additional information and are not simple 2D CNNs.

Table 3. Performance of 2D and 3D CNNs on Sports-1M dataset

Work	CNN type	Accuracy [%]	
		Video Hit@1	Video Hit@5
Karpathy et al., Single-Frame [33]	2D	59.3	77.7
Karpathy et al., Slow Fusion [33]	2D	60.9	80.2
Joe Yue-Hei Ng et al., Conv Pooling + Image and Optical Flow [77]	2D	72.4	90.8
Joe Yue-Hei Ng et al., LSTM + Image and Optical Flow [77]	2D	73.1	90.5
P3D ResNet [46]	(2+1)D	66.4	87.4
R(2+1)D, Image and Optical Flow [65]	(2+1)D	**73.3**	**91.9**
C3D [64]	3D		
Trained from scratch		60.0	84.4
With pre-training		61.1	85.2

Table 4. Performance of 2D and 3D CNNs on UCF101 Dataset

Work	CNN type	Accuracy [%] (3-fold)
Karpathy et. al, Slow Fusion and Transfer learning from Sports-1M [33]	2D	65.4
Joe Yue-Hei Ng et al., Conv Pooling + Image and Optical Flow [77]	2D	88.2
Joe Yue-Hei Ng et al., LSTM + Image and Optical Flow [77]	2D	88.6
R(2+1)D, Image and Optical Flow [65]	(2+1)D	97.3
P3D ResNet [46]	(2+1)D	88.6
I3D [7]	3D	**97.9**
C3D [64]	3D	90.4
ResNeXt-101 [25]	3D	94.5

Karpathy et al. [33] propose multiple approaches to help 2D CNNs take advantage of temporal information. They experiment with fusing the temporal dimension in the Single-Frame model (the baseline model) in three different ways: Late, Early, and Slow Fusion. The Early Fusion model integrates temporal information in the first convolutional layer by extending the filters of this layer to be of size $WxHxCxT$, where W and H represent the width and height of the frame, C is the number of channels and T is the temporal dimension. The Late Fusion model consists of two separate Single-Frame models which operate on frames which are 15 frames apart and the results are combined in the first fully-connected layer. The best results on the Sports-1M dataset were achieved with the Slow Fusion model which is a combination of the Early and Late Fusion models. However, the performance of the Slow Fusion model is close to the Single-Frame baseline model. One of the reasons why there is not a significant increase in accuracy might be that the temporal information is taken into account but it is collapsed after the first convolutional layer for the Early Fusion model and after the third convolutional layer for Slow Fusion. Karpathy et al. [33] also show that transfer learning can be beneficial. The Slow Fusion model pre-trained on Sports-1M then applied to UCF101 achieves a significant increase in accuracy compared to the model trained from scratch on UCF101, from 41.3% to 65.4%.

The approach of Joe Yue-Hei Ng et al. [77] is to compute the features of every frame and then to use a pooling layer across the temporal dimension. They experiment with multiple types of pooling layers (e.g., Average Pooling, Max Pooling) and different placement of these layers in the architecture of the network. They achieved overall better results by using the Conv Pooling architecture which uses Max Pooling across the temporal dimension in the last convolutional layer. Because this approach is time-invariant, they also explore the **Long Short-Term Memory** (**LSTM**) architecture which takes into account the order of the frames and combines it with the basic CNN.

Both [77] and [65] use, in addition to the frames which are RGB images, an input called *optical flow*. Optical flow is a representation of the estimated motion of objects across frames based on the changes in brightness. As expected, using the motion information given by the optical flow input increases the accuracy of the 2D CNNs on action recognition tasks. We report in Tables 3 and 4 their best results on the corresponding datasets.

When we refer to 3D CNNs throughout this article, we include the architectures that use 3D, decomposed ((2+1)D) and mixed (2D + 3D) convolutions. The architectures based on decomposed or (2+1)D convolutions split the 3D convolution into 2D convolution for the spatial dimensions followed by a 1D convolution for the temporal dimension. The architectures based on mixed convolutions use both 2D and 3D convolutions in their order of choice. These architectures are explained in more detail in Section 6. The accuracy in bold in Table 3 and Table 4 represents the highest accuracy among all networks on the corresponding dataset. The conclusion of this comparison is that on both UCF101 and Sports-1M, the 3D CNNs outperformed the 2D CNNs in terms of accuracy. While on UCF101 there is a significant difference in performance between 2D and 3D CNNs, on Sports-1M, the work of [77] is close with a difference of only 0.2% Video Hit@1 accuracy. This is due to the LSTM cells which improve the performance of the CNN by integrating frame order information.

5 Comparison between FPGAs, GPUs, and CPUs for 3D CNN Acceleration

FPGAs represent a flexible platform for **Deep Neural Network** (**DNN**) deployment which allows customization of the accelerators for different DNNs. This is an advantage over ASICs, which either have a generic architecture or are highly customized for a specific network. Generic architectures may not achieve the performance of customized architectures and custom FPGA-based accelerators can be designed and tested quickly and have lower cost compared to an ASIC. FPGAs can be reconfigured at run time using partial reconfiguration, reducing the configuration time and providing the possibility to reallocate resources over time. ASICs achieve lower power consumption, however, they do not provide the same flexibility as FPGAs. Compared to GPUs, FPGAs are typically more power efficient as well as being more flexible with the ability to implement custom data types. FPGAs are also particularly attractive to the real-time application domain of video classification as they can facilitate direct integration through chip-to-chip interfaces with video devices and can transfer data without the need of intervention from a host CPU. This leads to a significant reduction in system level latency, which is critical in real-time applications where decisions need to be made fast. This section compares the performance of FPGA 3D CNN accelerators to implementations on GPUs and CPUs which are directly comparable as they are reported in the surveyed articles.

Table 5 compares the performance in terms of throughput and power of 3D CNN accelerators implemented on FPGAs versus implementations on CPUs and GPUs. All these results are gathered from the surveyed articles; for this reason, some of the comparisons are incomplete. Table 6 lists the characteristics of the devices compared in Table 5. Table 5 reveals three trends: FPGAs achieve higher throughput than CPUs, GPUs have higher throughput than FPGAs, and FPGAs use less power than both CPUs and GPUs. For example, the 3D-VNPU [9] accelerator for 3D ResNet-18 achieves a 27x higher throughput than the CPU, but the GPU has 2x higher throughput than the FPGA design. In terms of power, the CPU and the GPU use 11.2x and 24.2x more power compared to the FPGA, respectively. Table 6 compares the respective devices in terms of technology and frequency. In this case, the FPGA is a Xilinx ZCU102 with 16 nm technology and running at 200 MHz, while the Intel E5-2680 v2 CPU has 22 nm technology and a frequency of 2.8 GHz, and the NVIDIA GTX TITAN X has 16 nm technology and runs at $\backsim 1GHz$. The technology of the device and the frequency impact power consumption. Note that for Sun et al. [57] the difference in power consumption between the FPGA, the CPU and the GPU is not as high as in the other cases; the

Table 5. Performance of 3D CNNs Accelerators on FPGAs, GPUs and CPUs

Work	3D CNN	Throughput				Power			
		FPGA vs CPU		FPGA vs GPU		FPGA vs CPU		FPGA vs GPU	
F-C3D [15]	C3D	–	–	–	–	1x	17.3x	1x	28.4x
F-E3D [13]	E3DNet	–	–	–	–	1x	3.7x	1x	4x
3D-VNPU [9]	3D ResNet-18	27x	1x	1x	2x	1x	11.2x	1x	24.2x
J. Shen et al. [53]	C3D	13.3	1x	1x	5.2x	1x	4.4x	1x	6.9x
J. Shen et al. [54]	C3D	17.8x	1x	1x	1.1x	1x	3.5x	1x	5.6x
Liu et al. [42]	C3D	8.2x	1x	1x	7.8x	1x	5.3x	1x	15.8x
Fan et al. [14]	C3D	–	–	–	–	1x	13.1x	1x	16.9x
HARFLOW3D [63]	C3D	–	–	–	–	–	–	1x	24.7x
Sun et al. [58]	R(2+1)D	1.6x	1x	1x	29.1x	–	–	1x	34.3x
Sun et al. (2) [57]	C3D	6.7x	1x	1x	2.2x	1x	1.2x	1x	2.3x
Sun et al. (2) [57]	R(2+1)D	7.6x	1x	1x	1.6x	1x	1.1x	1x	2.1x
FMM-X3D [61]	X3D-M	–	–	–	–	–	–	1x	9.9x
Xiang et al. [70]	R(2+1)D	8.7x	1x	1x	2x	1x	–	1x	25.7x

Table 6. Device Characteristics

Work	3D CNN	FPGA Name Technology, Frequency	CPU Name Technology, Frequency	GPU Name Technology, Frequency
F-C3D [15]	C3D	Xilinx ZC706 28 nm, 172 MHz	Intel i7-950 45 nm, 3.07 GHz	NVIDIA GTX 860M 16 nm, 1.02 GHz
F-E3D [13]	E3DNet	Intel Arria 10 SX660 20 nm, 150 MHz	Intel Xeon E5-2680 v2 22 nm, 2.8 GHz	NVIDIA TITAN X Pascal 16 nm, 1.53 GHz
3D-VNPU [9]	3D ResNet-18	Xilinx ZCU102 16 nm, 200 MHz	Intel E5-2680 v2 22 nm, 2.8 GHz	NVIDIA GTX TITAN X 16 nm, 1 GHz
J. Shen et al. [53]	C3D	Xilinx VUS440 20 nm, 200 MHz	Intel E5-2680 v2 22 nm, 2.8 GHz	NVIDIA GTX1080 16 nm, 1 GHz
J. Shen et al. [54]	C3D	Xilinx VCU118 14 nm, 200 MHz	Intel E5-2680 v2 22 nm, 2.8 GHz	NVIDIA GTX1080 16 nm, 1 GHz
Liu et al. [42]	C3D	Xilinx VC709 28 nm, 120 MHz	Intel i5-4440 22 nm, 3.10 GHz	NVIDIA TITAN X Pascal 16 nm, 1 GHz
Fan et al. [14]	C3D	Xilinx ZC706 28 nm, 200 MHz	Intel i7-950 45 nm, 3.07 GHz	NVIDIA TITAN X Pascal 16 nm, 1.53 GHz
HARFLOW3D [63]	C3D	Xilinx ZCU106 16 nm, 200 MHz	–	NVIDIA RTX 3090 8 nm, 1.7 GHz
Sun et al. [58]	R(2+1)D	Xilinx ZCU102 16 nm, 150 MHz	Intel Xeon E5-1650 14 nm, 3.6 GHz	NVIDIA GeForce GTX 1080 Ti 16 nm, 1.4 GHz
Sun et al. (2) [57]	C3D R(2+1)D	Xilinx ZCU102 16 nm, 150 MHz	Orin NX CPU 5 nm, 1.2 GHz	NVIDIA Jetson Orin NX 8 nm, 918 MHz
FMM-X3D [61]	X3D-M	Xilinx ZCU102 16 nm, 142 MHz	-	GeForce RTX 3090 8 nm, 1.7 GHz
Xiang et al. [70]	R(2+1)D	Xilinx Alveo U50 16 nm, 328 MHz	AMD Ryzen 3960X 7 nm, 3.8 GHz	NVIDIA RTX 3090 8 nm, 1.7 GHz

reason could be the advanced 5 and 8 nm technology and the lower frequencies (1.2 GHz for the CPU and 918 MHz for the GPU). This can also be seen in Figure 7 for the implementation of C3D. These are the only CPU and GPU points present in the cluster of FPGA results. Other than these, we observe where the performance of each type of devices lies – GPUs can provide high throughput but at a cost of high power consumption, CPUs consume less power but have a low throughput, while FPGAs have low power consumption, and generally better throughput than CPUs but lower than GPUs. However, J. Shen et al. [54] show that FPGAs can also be competitive to GPUs in terms of throughput.

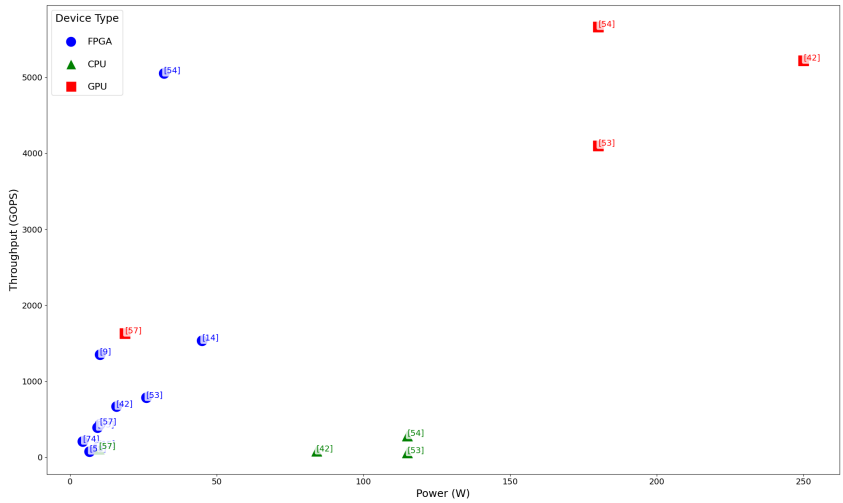


Fig. 7. Comparison between FPGAs, CPUs and GPUs in terms of throughput and power for the implementation of C3D.

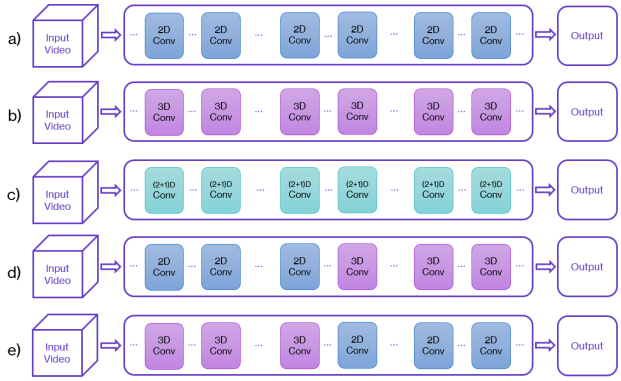


Fig. 8. CNN Architectures used for Video Classification. (a) 2D CNN. (b) 3D CNN. (c) (2+1)D CNN. (d) Mixed CNN with 3D Convolutions in right side layers*. (e) Mixed CNN with 3D Convolutions in left side layers*. *In this figure left side layers are top layers, the closest to the input, while right side layers refer to the bottom layers, closest to the output.

6 3D CNN Architectures

This section focuses on the architectures of state-of-the-art 3D CNNs. Figure 8 (a) shows a CNN based on 2D convolutions which can process an input video either frame by frame or by using the methods mentioned in Section 4. There are multiple types of 3D CNNs based on the types of convolutions used and they are shown in Figure 8 (b)–(e):

— 3D CNN: all convolutional layers use 3D convolutions (e.g., C3D, I3D) (Figure 8 (b))
  — Subtypes of 3D convolutions include the following:
    – 3D depth-wise Convolution: keeps the input channels separate and applies a kernel on each instead of applying the kernel across all channels simultaneously

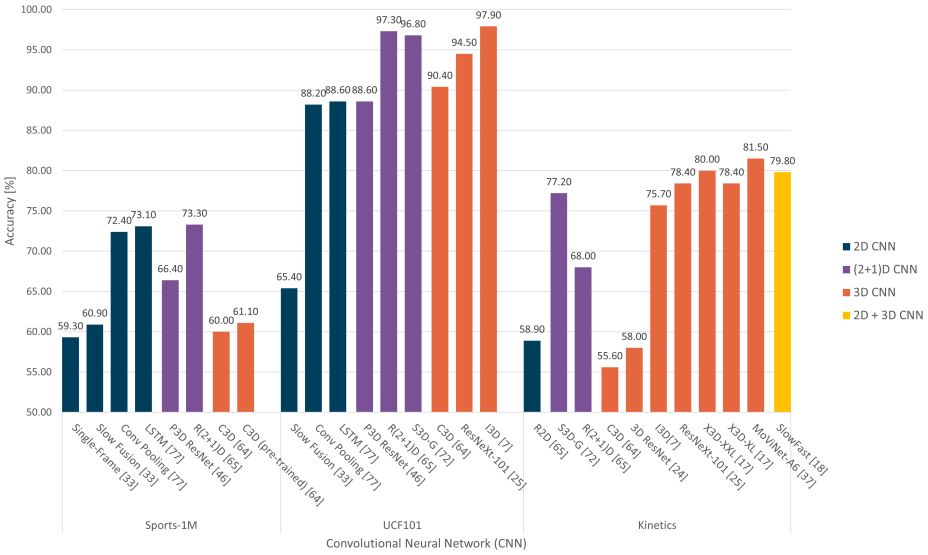


Fig. 9. Comparison of 2D, 3D, (2+1)D and 2D + 3D CNNs on 3 Action Recognition Datasets (Sports-1M, UCF101 and Kinetics). The accuracy reported is the Video Hit@1 Accuracy for Sports-1M and Kinetics datasets and the 3-fold accuracy for UCF101.

  - 3D point-wise Convolution: uses a kernel size of 1 x 1 x 1 (H x W x T where H and W represent the spatial dimensions (height and width of the feature map), T represents the temporal dimension (video frame)) and combines feature information across all channels for an individual voxel
- (2+1)D CNN: 3D convolutions are decomposed into 2D and 1D convolutions (e.g., R(2+1)D, P3D ResNet) (Figure 8 (c))
- Mixed CNN (as introduced by [65]): 3D or (2+1)D convolutions are used in the early layers and 2D convolutions in the other layers (e.g., Bottom-Heavy-I3D [72], Top-Heavy-I3D [72], SlowFast [18]) (Figure 8 (d)–(e))

Table 7 summarizes and compares the architectures and performance of the state-of-the-art 3D CNNs. The accuracy in bold represents the maximum accuracy among all architectures on the corresponding dataset. As defined in Section 4, the metrics Video Hit@1 and Video Hit@5 represent the Top-1 and Top-5 accuracies, the 3-fold metric is specific to the UCF101 dataset and it is the mean Top-1 accuracy on 3 predefined splits of the dataset, and Clip Hit@1 represents the Top-1 accuracy on a sampled clip from the input video.

Figure 9 shows a comparison between different types of 3D CNNs and 2D CNNs on 3 different action recognition datasets: Sports-1M, UCF101 and Kinetics. There is a significant performance difference on UCF101 and Kinetics between 2D and different versions of 3D CNNs; however, the accuracy of 2D CNNs obtained on Sports-1M is close to the one of a (2+1)D CNN. Most of the leading work on the other two datasets has not been tested on Sports-1M which is a larger and more challenging dataset.

Among these state-of-the-art 3D CNNs, C3D is one of the oldest and is widely adopted as a comparison point between different hardware accelerators. It is a fairly shallow network of 8 convolutional layers which makes it easy to train and deploy on hardware, but it also achieves lower accuracy especially on larger datasets such as Kinetics.

Table 7. 3D CNN Architectures

Name	Conv Type	Structure	Model Size	Dataset	Accuracy [%]			
					Clip Hit@1	Video Hit@1	Video Hit@5	3-fold
C3D [64], 2015	3D	8 Convolution layers 5 Max Pooling Layers 2 Fully-Connected Layers 1 Softmax Output Layer	321MB [13]	Sports-1M UCF101 Kinetics	46.1 79.87 [13] –	61.1 – 55.6 [65]	85.2 – 79.1 [65]	– 90.4 –
P3D ResNet [46], 2017	(2+1)D	199 layers Pre-trained on Imagenet	261 MB	Sports-1M UCF101	**47.9** 84.2 [13]	66.4 –	87.4 –	– 88.6
I3D [7], 2017	3D	Two Input Streams: RGB and Optical Flow; Pre-trained on Imagenet and Kinetics; 72 layers	25M params 107.89 GFLOPS [72]	UCF101 HMDB51 Kinetics Kinetics	– – – –	– – 75.7 [17] 74.1 [72]	– – 92.0 [17] 91.6 [72]	**97.9** 80.7 – –
3D ResNet [24], 2017	3D	18 or 34 layers	–	Kinetics	–	58.0	81.3	–
ResNeXt-101 [25], 2018	3D	101 layers Pre-trained on Kinetics	365MB [13]	UCF101 HMDB51 Kinetics	87.7 [13] – –	– – –	– – 78.4*	94.5 70.2 –
S3D-G [72], 2018	(2+1)D	Based on Inception-V1 [59] RGB + Optical Flow Pre-trained on ImageNet	11.56M params 71.38 GFLOPS	Kinetics UCF101	– –	77.2 –	93.0 –	– 96.8
R(2+1)D [65], 2018	(2+1)D	18 or 34 layers	33.3M params	Kinetics Sports-1M UCF101	**56.8** – –	68.0 **73.3** –	– **91.9** –	– – 97.3
E3DNet [13], 2019	3D	Temporal convolutions Depth-wise convolutions Point-wise convolutions Pre-trained on Kinetics	8.6 MB	UCF101	**85.17**	–	–	–
X3D [17], 2020 X3D-XL X3D-XL X3D-XXL	3D	Expanded from MobileNet [29]	 1452 GFLOPS 11M params 1452 GFLOPS 11M params 5823 GFLOPS 20.3M params	 Kinetics Kinetics-600 Kinetics	 – – –	 78.4 81.9 80.0	 93.6 95.5 94.5	 – – –
[38], 2019 3D-SqueezeNet 3D-ShuffleNetV1 3D-ShuffleNetV2 3D-MobileNetV1 3D-MobileNetV2	 3D Mixed Mixed Mixed Mixed	3D extensions of popular 2D CNNs: SqueezeNet [30], MobileNet [29], ShuffleNet [78], MobileNetV2 [50] and ShuffleNetV2 [43]	 728 MFLOPS 2.15M params 393 MFLOPS 4.78M params 360 MFLOPS 6.64M params 454 MFLOPS 14.10M params 446 MFLOPS 3.12M params	 Kinetics-600 Kinetics-600 Kinetics-600 Kinetics-600 Kinetics-600	 – – – – –	 40.52 56.84 55.17 48.53 50.65	 – – – – –	 – – – – –
MoViNet [37], 2021 MoViNet-A4 MoViNet-A5 MoViNet-A6 	3D	Expanded from MobileNet [29]	 105 GFLOPS 5.9M params 281 GFLOPS 15.7M params 386 GFLOPS 31.4M params 386 GFLOPS 31.4M params	 Kinetics Kinetics Kinetics Kinetics-600	 – – – –	 80.5 80.9 **81.5** **83.5**	 94.5 94.9 **95.3** **96.2**	 – – – –
SlowFast [18], 2019	Mixed	Based on ResNet-101 [26] and Non-local neural networks [67]	7020 GFLOPS	Kinetics Kinetics-600	– –	79.8 81.8	93.9 95.1	– –

*Hit@1 and Hit@5 average accuracy on the Kinetics dataset.

P3D ResNet [46], 3D ResNet [24], R(2+1)D [65] and ResNeXt-101 [25] are all based on the ResNet [26] architecture. The P3D ResNet [46] is a Pseudo 3D CNN which expands ResNet152 [26] and instead of using 3D convolutions, applies the (2 + 1)D method. This means that the 3D convolution is split between a 2D convolution to cover the spatial domain and a 1D convolution to cover the temporal domain. The authors state that the advantage of this method is that it reduces the size of the model significantly and it can benefit from 2D CNN pre-training on image datasets. Compared to P3D ResNet [46] which uses a sequence of 2D and 1D convolutions, 3D ResNet [24] is based on 3D convolution and pooling layers.

ResNeXt-101 [25] is a deeper version of 3D ResNet and has a slightly different architecture of the residual building block. While in other work [24] the evaluated 3D ResNet models have 18 and 34 layers, ResNeXt has 101 layers. In comparison to a basic bottleneck ResNet block, ResNeXt [71] uses cardinality which is the technique of splitting the feature map in groups and performing group convolutions. It has been shown in [24] that this method increases the performance of 2D models and is more efficient than increasing the depth of the networks. Hara et al. [25] show that 3D ResNet with 18 layers tends to overfit on UCF101, HMDB51, and ActivityNet when it is trained from scratch (no pre-training), but it does not overfit on the Kinetics dataset. This indicates that larger datasets are preferred for training 3D CNNs. The authors also highlight that pre-training on a large dataset (Kinetics, in this case) leads to competitive results on smaller datasets (such as UCF101).

Tran et al. [65] introduce R(2+1)D, a network which, similar to P3D ResNet, is based on the decomposition of 3D convolution into separate 2D and 1D convolutions. Compared to P3D ResNet, R(2+1)D uses only one type of residual building block in all layers and it contains only 34 layers.

Targeting FPGA acceleration of 3D CNNs, Fan et al. [13] propose a more optimized 3D CNN, called E3DNet, as well as an accelerator design, F-E3D, which is discussed in Section 7. The main building block of **E3DNet is the Bottleneck Residual Block** (**3D BRB**) which is based on depth-wise and point-wise convolutions. Compared to a regular 3D convolution, the depth-wise convolution keeps the input channels separate and applies a kernel on each one, hence there is no channel accumulation. When the input volume is separated by the channel dimension, C blocks (where C is the number of channels) of H x W x Kt are generated (where H and W represent the height and width of the input feature map, respectively, and Kt is the temporal kernel size). 2D depth-wise convolutions are applied to each of the C groups generating Kt feature maps which are accumulated per group in a final output frame. The 3D BRB makes E3DNet a reduced size model while maintaining competitive accuracy. The authors report an accuracy of 85.17% for E3DNet on the UCF101 dataset, which is between the closest most accurate networks, ResNeXt-101 (87.7%) and P3D ResNet (84.2%). However, the size of the E3DNet model is only 8.6 MB while ResNeXt-101 is 365 MB and P3D ResNet is 261MB. They report the Clip Hit@1 accuracy which means that only one clip per video is used for prediction ([13] define the clip as a 16 frame random sample from the input video). There are two proposed versions of 3D BRB: 3D-1 BRB and 3D-3 BRB, the difference between them being the ordering of different types of convolutions. The authors' analysis and comparison of 3D-1 and 3D-3 BRB reveal that 3D-1 has a lower computational load and a lower memory demand, which facilitates the deployment of the network on hardware.

I3D [7], also called Two-Stream Inflated 3D ConvNet, is based on the Inception-V1 [59] network trained on the ImageNet [10] database. I3D has 19 layers out of which 9 layers are inception layers. These inception layers contain 6 convolution layers and one max pooling layer. In total, the depth of the network is 72 layers and it achieves the highest accuracy on the UCF101 dataset.

S3D [72] is based on I3D [7] but the 3D convolutions are replaced with (2+1)D convolutions. The most accurate model in [72] is S3D-G which, compared to S3D, contains a feature gating layer for improving performance. This feature gating layer adds more FLOPS, from 66.38 to 71.38

GFLOPS, but the increase in accuracy is over 2%. Overall, S3D-G has fewer parameters, is more computationally efficient and achieves a comparable accuracy to I3D; however, the feature gating layer needs to be taken into account when designing hardware accelerators for this type of network.

Köpüklü et al. [38] target resource efficient 3D CNNs in terms of number of parameters and number of FLOPS required. They implement 3D versions of several popular 2D CNNs (SqueezeNet [30], MobileNet [29], ShuffleNet [78], MobileNetV2 [50] and ShuffleNetV2 [43]) and they vary their complexity by adjusting the number of channels. Their analysis shows that reducing the computational complexity significantly affects the accuracy of shallower networks such as 3D-SqueezeNet and 3D-MobileNetV1. 3D-ShuffleNet, 3DShuffleNetV2 and 3D-MobileNetV2 are deeper architectures with higher accuracy. However, even for 3D-MobileNetV2 (561 MFLOPS) which achieves 81.6% accuracy on UCF101, when the number of channels is 0.7x lower (the complexity being reduced to 325 MFLOPS), the accuracy drops to 77.32%. The accuracy reported on UCF101 is the one obtained on split-1, not 3-fold as for the other CNNs, hence we do not include these results in Table 7.

Feichtenhofer [17] expands a compressed version of MobileNet [29] into multiple 3D CNNs. Compared to other 3D models resulted from expanding 2D models across the temporal dimension, they expand the spatial and temporal dimensions, the width (number of channels) and depth (number of layers) in order to improve the computational complexity/accuracy tradeoff. One of the main findings of this work is that a high spatio-temporal resolution compensates for a small channel dimension and this leads to an efficient 3D CNN for video classification. They classify the expanded models in three categories based on complexity: small, medium and large. Their best accuracy on Kinetics was achieved with the large X3D-XXL model but with a significant increase in GFLOPS. The corresponding values can be seen in Table 7.

Similarly to X3D [17], MoViNets [37] are built on MobileNetV3 [28] which is extended to handle 3D input data. Kondratyuk et al. [37] use the TuNAS framework [3] which does **Neural Architecture Search** (**NAS**) to generate a set of computation and memory efficient 3D CNN architectures. Starting from MobileNetV3 and a search space across width, depth, input resolution, and frame rate, MoViNets A0-A6 are generated. Their largest network, MoViNetA6 achieves a 3.1% increase in Video Hit@1 on the Kinetics dataset and a 1.6% increases on the Kinetics-600 dataset compared to X3D-XL, while also reducing the number of GFLOPS 3.7x. However, MoViNetA6 has almost 3x the number of parameters compared to X3D-XL.

The SlowFast Network [18] is composed of a Slow pathway to capture spatial semantics which has low frame rate, and a Fast pathway to capture motion which has high frame rate. Feichtenhofer et al. present multiple SlowFast networks with different backbones such as ResNet-50 [26], ResNet-101 [26] and Non-local neural network [67]. Their best results on Kinetics and Kinetics-600 are achieved by a combination between ResNet-101 [26] and Non-local neural network [67]. Compared to the most accurate work in Table 7 - MoViNet-A6, SlowFast is less accurate while also having around 18x more GFLOPS.

To conclude this section, there is a wide range of available 3D CNNs for which we have identified several commonalities. First of all, most of the 3D networks have been expanded from popular 2D CNNs. The most common backbones being ResNet [26] and MobileNet [29]. Second, the 3D CNNs can be grouped by convolution types: 3D, (2+1)D and Mixed Convolutions. These are important observations for designing hardware accelerators, especially general designs that can support multiple 3D CNN architectures. For example, one might choose to target the networks with ResNet backbones or only the networks which use 3D convolutions; therefore, this dictates what kind of layers need to be supported in hardware. 2D CNN hardware accelerators can also be extended to handle (2+1)D convolutions, leading to a common accelerator architecture for both image classification (2D) and action recognition (3D) CNNs. However, there are also specific

characteristics for different networks such as the feature gating layer of S3D-G [72] or the 3D BRB module of E3DNet [13] which can require additional hardware blocks. Another takeaway is that a reduced number of FLOPS and a reduced number of parameters can significantly decrease the accuracy (see [38] in Table 7). Balancing the accuracy, the number of FLOPS and the number of parameters is key to achieving high performance accelerators, especially when it comes to resource limited devices such as FPGAs. Some of the surveyed articles do not provide these numbers, but on the Kinetics dataset we can identify the network with the best tradeoff: MoViNet-A6 [37] with 81.5% accuracy, 386 GFLOPS and 31.4M parameters. If one prefers to trade a small decrease in accuracy for lower FLOPS and number of parameters, MoViNet-A4 and MoViNet-A5 are also good candidates. This can be noticed as well in Figure 1 and Figure 2 as MoViNet A4-A6 are part of the Pareto front. MoViNet-A4 has higher accuracy compared to the closest contender, X3D-XXL [17] (80.5% versus 80.0%) while there is a remarkable difference in complexity: 105 versus 5823 GFLOPS and 5.9M versus 20.3M parameters, respectively.

In this section, we have described several 3D CNN architectures. In the next section, we provide details on 3D CNNs implemented on FPGAs.

7 FPGA-based 3D CNN Accelerators

This section summarizes the challenges of 3D CNN hardware acceleration and analyzes state-of-the-art FPGA-based accelerators for 3D CNNs and design optimization techniques.

7.1 Challenges of 3D CNN Hardware Acceleration

The design techniques for 3D CNNs are based on the same ideas as those used for 2D CNNs, but they need to be adapted for the following challenges of 3D CNN hardware acceleration:

Increased computational complexity: 3D CNN FPGA-based accelerators need to be able to process 3D volumes which implies handling a larger amount of computation than 2D CNN accelerators and to support efficient 3D, (2+1)D or mixed convolutions.

Memory management: 3D CNN FPGA-based accelerators need to ensure efficient data processing, movement and storage. The additional temporal dimension complicates the architecture design as the limited FPGA resources may not be able to accommodate the large number of parameters, the large input volumes and the intermediate results. The R(2+1)D article [65] shows that the R2D CNN (based on a ResNet architecture) has 11.4 M parameters, while the 3D CNN, R(2+1)D, has 33.3M parameters. This is an example of the difference between 2D and 3D CNNs with only 18 layers; the difference can be significantly higher for deeper networks. The goals of 3D CNN accelerators in terms of memory management are to capture spatio-temporal memory access patterns, exploit locality, avoid data replication and minimize the number of off-chip memory accesses.

Parallelism: Parallelism in 3D CNN accelerators needs to be more aggressive in order to minimize latency, maximize throughput and achieve comparable performance to 2D CNN accelerators because they need to process both spatial and temporal dimensions, while dealing with the computational overhead of the extra dimension.

The analysis of C3D [53], a popular 3D CNN covered in Section 6, shows that the 3D convolutions take up over 99.9% of the total GFLOPS and over 97.3% of the computation time. The fully-connected layers take less than 1% of the computation time but demand storage for over 70% of the weights. Another important conclusion is that a large amount of data needs to be transferred between layers (e.g., the convolution and pooling outputs where there are multiple 3D structures corresponding to the convolution of the input video with each convolution kernel). In C3D, this intermediate data has a size of 356.7 MB while the weights are 66.5 MB. This analysis is useful for the design of hardware accelerators as it provides hints to where the design should focus on accelerating computation and where data storage management and efficient data movement are important.

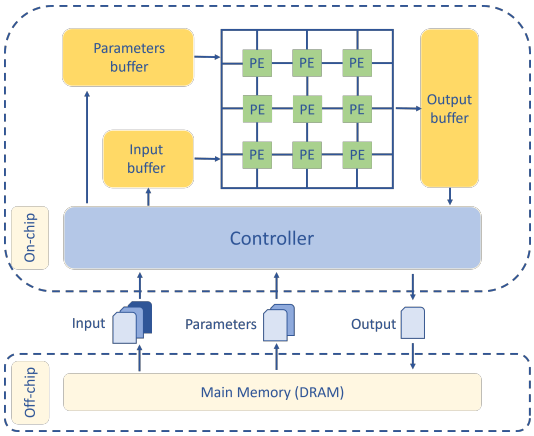


Fig. 10. General MPE architecture of an FPGA-based neural network accelerator.

We compare the accelerators in terms of resource utilization, latency, throughput, and power in Tables 8 and 9. We also compare them in terms of power and resource efficiency for the C3D model. Resource utilization is important for determining on which FPGA the design can fit. Latency and throughput are the main metrics for evaluating the performance and suitability of the accelerators for specific applications. In the context of this survey, a latency-oriented accelerator ensures fast video classifications (typically ms), while a high throughput accelerator ensures fast processing of large volumes of data during a given time frame (typically operations/s). Latency is critical for real-time video processing applications such as real-time video analytics and throughput is essential for applications that handle a high number of videos per second such as video transcoding. Power consumption is also an important metric as it is one of the main advantages of FPGAs over CPUs and GPUs, making them suitable for edge applications and more efficient and sustainable in areas such as high performance computing. In Tables 8 and 9, the "3D CNN" column also mentions number of parameters, number of layers and the size of the input (e.g., 16x112x112x3 stands for 16 frames, 112 height, 112 width, and 3 channels) where reported.

7.2 Typical Architectures of FPGA DNN Accelerators

There are two general architectures, **Matrix of Processing Elements** (**MPE**) and Streaming, with the MPE architecture being more common. An MPE architecture, shown in Figure 10, typically consists of a set of **Processing Elements** (**PEs**), data buffers (input, output and parameters) and a controller. The controller is responsible for bringing the data from the off-chip memory and scheduling the execution of layers onto the PEs. For each layer, the input of the layer and its parameters (e.g., weights) that are stored in off-chip memory need to be brought on-chip and stored in the corresponding input and parameter buffers. The output of each layer also has to be transferred back to off-chip memory, if it does not fit in the on-chip memory. The controller which controls the data movement and the PEs is, in some cases, managed by a CPU which provides custom instructions, or the CPU acts like the controller itself. For example, because HARFLOW3D [63] can generate different designs with different computing modules based on the input 3D CNN, their control is done through a CPU and each computing module can be activated through a custom instruction. The CPU also allows runtime reconfigurability; the parameters of the computation nodes (such as kernel sizes and padding) can be adapted from layer to layer and some hardware modules can be bypassed if not needed. Besides the array of PEs, designs such as HARFLOW3D [63] and 3D-VNPU [9] may include specialized modules for layers such as fully-connected or pooling.

In contrast, fpgaHART [62] and FMM-X3D [61] implement streaming architectures. Instead of scheduling the execution of layers on shared computing elements, each layer has a computing unit assigned, and the accelerator is data driven. In this architecture, a unit corresponding to a specific layer starts computing as soon as the previous layer's computing unit produces a result.

For both MPEs and streaming architectures, we classify FPGA-based accelerators into three main categories: **Specialized designs** which are tailored for a certain 3D CNN architecture, **Unified accelerators** which support both 2D and 3D CNNs, and **Toolflows** which can generate hardware accelerators for a wider range of 3D CNNs. These are described in the following subsections.

7.3 Specialized Designs

7.3.1 F-E3D. F-E3D [13] is the FPGA accelerator designed for E3DNet (described in Section 6). The architecture of F-E3D is based on a computational unit; sliding window, pooling and ReLU modules; and several buffers for data caching. The computational unit consists of multiple PEs which, in turn, are made of **Processing Units** (**PU**). A PU contains multipliers and an adder tree. The computational flow starts with temporal convolution, moves on to 3D depth-wise convolution and ends with point-wise convolution. Finally, the output is transferred back to off-chip memory. Three main strategies have been used to make F-E3D [13] an efficient accelerator: fused 3D BRB, online blocking, and kernel reuse.

One particular aspect of F-E3D, besides being tailored for the main building block of E3DNet, 3D BRB, is that the fused 3D Bottleneck Residual Block is optimized as a whole module, instead of attempting to accelerate each layer (temporal, depth-wise and point-wise convolution layers) separately, and the intermediate data needed within the block is cached in on-chip memory, facilitating data reuse. However, this requires a large amount of on-chip memory. Online blocking helps with memory management as discussed, along with kernel reuse, in Section 7.6.

7.3.2 Binary 3D CNN (3D AND-Net) Hardware Accelerator. Li et al. [41] implement a 3D CNN with binary weights and activations and design a tailored accelerator. The binary 3D CNN, called 3D AND-Net, is inspired by XNOR-Net [47] which is a binary 2D CNN. Predictably, a binarized 3D CNN has low resource utilization as summarized in Table 8. The architecture of the network contains six convolutional layers and a fully-connected layer, and it achieves an accuracy of 89.2% on KTH [52].

In order to maximize data reuse and optimize the convolution operation, Li et al. [41] use a method called O2M which implies that the input data is read only once; therefore there is no overlapping in reading data from the off-chip memory. This helps decrease latency and power consumption associated with these fetches of the input data. The hardware accelerator for 3D AND-Net with only 7 layers cannot fit all intermediate results in on-chip memory on the target FPGA so these are all stored in the off-chip memory. Advantages of this accelerator are that it is flexible, it accepts layers with different parameters, and it can process wider and deeper 3D AND-Nets.

7.3.3 I3D Hardware Accelerator. This article [36] is one of the works (such as HARFLOW3D) that tackle the problem of hardware acceleration for deeper and more complex 3D CNNs than C3D, namely the I3D model. As seen in Table 7, I3D is more accurate on both UCF101 and Kinetics datasets, but it also has 72 layers compared to C3D which has only 16 layers.

Khan et al. [36] analyze I3D and propose several improvements in terms of quantization, data tiling and parallelization. They propose post-training quantization of weights and feature maps and they show that by going from 32 bit floating point precision to 8 bit integers the drop in memory requirements is significant (e.g., for an inception layer which requires storage for parameters of over 8000 KB when using 32b floating point, the memory demand when using 8b integers decreases to a little over 2000 KB) while the drop in accuracy is only 2%. They also investigated optimizations

Table 8. FPGA-based 3D CNN Accelerators

Name, Year	3D CNN	FPGA	Precision	Resource Utilization	Throughput [GOPS]	Latency [ms]	Power [W]
F-C3D [15] (2017)	C3D	Xilinx ZC706 (172 MHz, 28 nm)	16 bit fixed point	LUT: 90,281 FF: 114,983 BRAM: 472 DSP: 810	142 [42]	5,425	9.7
F-E3D [13] (2019)	E3DNet 16x112x112x3	Intel Arria 10 SX660 (150 MHz, 20 nm)	32 bit floating point	ALM: 113,828 M20K: 1,578 DSP: 1,584	–	35.3	36
3D-VNPU [9] (2021)	C3D 3D ResNet-18	Xilinx ZCU102 (200 MHz, 16 nm)	8 bit fixed point	LUT: 192,000 BRAM: 712 DSP: 1024	1,353 1,210	– 15.93	10.2 10.2
J. Shen et al. [53] (2018)	C3D	Xilinx VC709 (150 MHz, 28 nm) Xilinx VUS440 (200 MHz, 20 nm)	16 bit fixed point	LUT: 242,000 FF: 286,000 BRAM: 1,508 DSP: 1,536 LUT: 209,000 FF: 285,000 BRAM: 1,476 DSP: 1,536	430.7 784.7	89.4 49.1	25 26
J. Shen et al. [54] (2019) (updated)	C3D	Xilinx VCU118 (200 MHz, 14 nm)	16 bit fixed point	LUT: 560,000 URAM: 320 BRAM: 3,392 DSP: 5,184	5,054	33.7	32
Liu et al. [42] (2019)	C3D	Xilinx VC709 (120 MHz, 28 nm)	16 bit fixed point	LUT: 272,734 FF: 434,931 BRAM: 391 DSP: 3,595	667.7	115.5	15.8
Fan et al. [14] (2018)	C3D 16x112x112x3	Xilinx ZC706 (200 MHz, 28 nm)	mantissa: 15 bit exponent: 8 bit sign: 1 bit BFP	LUT: 82,849 Register: 195,674 BRAM: 480 DSP: 780	–	29.8* (*per frame)	9.9
Fan et al. (2) [12] (2021)	C3D	Intel Arria GX1150 (220 MHz, 20 nm)	mantissa: 8 bit exponent: 4 bit sign: 1 bit Static BFP	ALM: 263,569 Register: 901,673 M20K: 2,334 DSP: 1,345	1,536	93.95	45
Tian et al. [60] (2020)	C3D 16x112x112x3	Xilinx VC707 (160 MHz, 28 nm)	8 bit fixed point	BRAM: 260.5 DSP: 2,688	714	107.9	–
Li et al. [41] (2022)	3D AND-Net	Xilinx PYNQ-Z2 (100 MHz, 28 nm)	1 bit	LUT: 21,727 BRAM: 140 DSP: 14	22.4	–	1.6
HARFLOW3D [63] (2023)	C3D 27 layers 78.41M params 16x112x112x3 R(2+1)D-18 82 layers 33.41M params 16x112x112x3 R(2+1)D-34 154 layers 63.72M params 16x112x112x3 X3D-M 396 layers 3.82M params 16x256x256x3	Xilinx ZCU102 (200 MHz, 16 nm) Xilinx VC709 (150 MHz, 28 nm) Xilinx ZCU102 (200 MHz, 16 nm) Xilinx VC709 (150 MHz, 28 nm) Xilinx ZCU102 (200 MHz, 16 nm) Xilinx VC709 (150 MHz, 28 nm) Xilinx ZCU102 (200 MHz, 16 nm) Xilinx VC709 (150 MHz, 28 nm)	16 bit fixed point 16 bit fixed point 16 bit fixed point 16 bit fixed point	BRAM: 1,282 DSP: 2,432 BRAM: 1,861 DSP: 3,519 BRAM: 1,232 DSP: 2,442 BRAM: 1,899 DSP: 3,524 BRAM: 1,234 DSP: 2,410 BRAM: 1,602 DSP: 3,501 BRAM: 1,001 DSP: 1,348 BRAM: 2,312 DSP: 3,225	393.37 424.14 173.91 185.13 184.29 206.39 43.78 120.38	98.15 91.03 48.99 46.02 70.05 62.55 155.07 56.14	9.44 – – – – – – –
Khan et al. [36] (2023)	I3D 12M params 64x224x224x3	Xilinx VC709 (200 MHz, 28 nm)	32b floating point 8b integer	BRAM: 588 DSP: 3,456 LUT: 589,152 BRAM: 2,322	684 1287	178 96	26 15

Table 9. (Table 8 Continued) FPGA-based 3D CNN Accelerators

Name, Year	3D CNN	FPGA	Precision	Resource Utilization	Throughput [GOPS]	Latency [ms]	Power [W]
Sun et al. [58] (2020)	C3D	Xilinx ZCU102 (150 MHz, 16 nm)	16 bit fixed point	LUT: 148,000 FF: 76,000 BRAM: 912 DSP: 1,215	79.1	487	6.7
	R(2+1)D			LUT: 148,000 FF: 76,000 BRAM: 912 DSP: 1,215	111.7	234	6.7
Sun et al. (2) [57] (2024)	C3D	Xilinx ZCU102 (150 MHz, 16 nm)	16b fixed point	LUT: 127,000 FF: 63,000 BRAM: 631.5 DSP: 1,909	435.3	177.44	10.17
			8b integer	LUT: 131,000 FF: 105,000 BRAM: 388.5 DSP: 1,651	739.8	54.07	7.85
	R(2+1)D		16b fixed point	LUT: 127,000 FF: 63,000 BRAM: 631.5 DSP: 1,909	404.9	201.38	10.18
			8b integer	LUT: 131,000 FF: 105,000 BRAM: 388.5 DSP: 1,651	695.2	66.99	7.92
fpgaHART [62] (2023)	C3D 27 layers 78.41M params 16x112x112x3	Xilinx ZCU102 (160 MHz, 16 nm)	16 bit fixed point	BRAM: 1,669 DSP: 1,298	130.84	–	–
	R(2+1)D-18 82 layers 33.41M params 16x112x112x3			BRAM: 1,425 DSP: 1,669	39.59	–	–
	R(2+1)D-34 154 layers 63.72M params 16x112x112x3			BRAM: 1,534 DSP: 1,675	34.26	–	–
	X3D 396 layers 3.82M params 16x256x256x3			BRAM: 962 DSP: 2,128	85.96	–	–
FMM-X3D [61] (2023)	X3D-M 244 layers 3.76M params 16x256x256x3	Xilinx ZCU102 (142 MHz, 16 nm)	16 bit fixed point	DSP: 2,142	119.83	–	26
Xiang et al. [70] (2024)	R(2+1)D-18 31.5M params 16x112x112x3	Xilinx Alveo U50 (328 MHz, 16nm)	8 bit fixed point	DSP: 4,620	1,511	38.98	13.5
Khan et al. [35] (2024)	C3D	Xilinx VC709 (200 MHz, 28 nm)	-	LUT: 547,564 BRAM: 808.5	824	86	–
	I3D			LUT: 547564 BRAM: 911.4	1,268	70	–
	R(2+1)D			LUT: 547,564 BRAM: 1,014	389	41	–
Yang et al. [74] (2023)	C3D	Xilinx XCZU15EG (100 MHz, 16 nm)	8 bit integer	LUT: 94,804 FF: 59,947 BRAM: 456 DSP: 1,221	209.4	–	4.35

to fit their design onto an FPGA with limited resources. We discuss their approach to data tiling and parallelization in Section 7.6.

The final proposed accelerator architecture for I3D is based on multiple PUs, each containing an array of PEs, such as F-E3D [13] or 3D-VNPU [9]. One contribution of this work is **Design Space Exploration** (**DSE**) with regards to the size of the PE array. Experiments are conducted to determine the effect of the array size on latency, throughput and resource utilization. For 32b floating point precision I3D, the best configuration is an 8x8 PE array, resulting in a throughput of 684 GOPS and a latency of 178 ms. For 8b integer precision I3D, the best configuration is a 16x16 PE array, resulting in a throughput of 1287 GOPS and a latency of 96 ms. These results are summarized in Table 8.

7.3.4 FMM-X3D. Toupas et al. [61] follow the fpgaConvNet [66] paradigm and extend it to map the X3D-M model to FPGA. fpgaConvNet is a framework for deploying 2D CNNs on FPGAs based on which both HARFLOW3D [63] and fpgaHART [62] toolflows have been developed (see Section 7.5). The fpgaConvNet framework generates designs with a streaming architecture where each layer has its own computing unit. The main advantage of this architecture is that the computing unit is optimized for a specific layer and parallelism can be explored independent of other layers. However, the disadvantage of having a custom hardware block for each layer is excessive resource utilization. For large networks, such as X3D-M with over 200 layers, the design can require many resources and it may be impossible to fit the design on one device. This work [61] proposes a solution, namely partitioning the model execution: creating a separate bitstream for each partition and sequentially loading each partition onto the board. The authors state that the reconfiguration cost can be compensated by increasing the batch size (the input data to be processed by each partition).

7.3.5 R(2+1)D Accelerator. Xiang et al. [70] developed a custom accelerator for R(2+1)D. The architecture of the accelerator includes independent computing modules containing PE arrays for 2D and 1D convolutions, buffers for weights and feature maps, a loop controller which fetches data from the buffers and feeds it to the computing modules, and a pooling and element-wise operation module. A host CPU is used to update the configuration register with the corresponding parameters of the convolutional layer. DSE is used to maximize the throughput while respecting the resource constraints of the targeted FPGA. This is done by determining the design parameters which maximize hardware parallelism and the ones which dictate the on-chip buffer sizes. The authors apply tiling to the output channel because their analysis shows that this method does not involve any overlapped data and thus ensures continuous off-chip memory access. Other design methods they explore are loop unrolling and loop interchange which maximize the computing efficiency of the PE arrays for both the 1D and 2D computing modules.

7.4 Unified Accelerators for 2D and 3D CNNs

7.4.1 3D-VNPU. 3D-VNPU [9] is an FPGA-based flexible accelerator suitable for both 2D and 3D CNNs. In order to reduce the computational complexity of the networks, especially for 3D CNNs where the number of **multiply-accumulate** (**MAC**) operations significantly exceeds those of 2D CNNs, 3D-VNPU relies on the Winograd algorithm. The Winograd method reduces the number of multiplications by computing convolutions in a different manner, but it increases the number of additions (see Section 7.6). However, additions need fewer resources. With their **Flexible Winograd-based Decomposition Method** (**FWDM**) the authors can cut the number of multiplications approximately 60%–70%. FWDM supports different convolution dimensions, kernel sizes and strides which makes the accelerator highly scalable.

FWDM employs two types of decomposition: activation and filter decomposition. Activation decomposition is applied only when the stride is greater than 1, and it splits the activation volume in 2^n blocks, where n is the dimensionality of the stride (for example, if stride is (2, 2, 2), n is 3).

The advantage of this is that there is no data overlapping and no additional storage required. The filter/kernel decomposition method divides the convolution kernels in smaller blocks and works for strides 1 and 2. Compared to other methods such as [9], the blocks do not need to be padded with zeros to fit the Winograd unit because multiple types of Winograd units are adopted and the kernels are split correspondingly. Another advantage of 3D-VNPU is that it supports convolution strides of one and two. Other accelerators based on Winograd such as [54] only support strides of one, and are therefore not suitable for state-of-the-art 3D CNNs such as I3D and 3D ResNet which require a stride of 2.

3D-VNPU consists of clusters which contain 2D/3D computing engines, a controller, a ReLU and Pooling Unit, a **Data Reorganization Unit** (**DRU**) and memory. The cluster has a configurable controller which is used to fetch the activations and weights, and store them in the corresponding buffers. These buffers feed the data to the cores which are the 2D/3D compute engines. The DRU decomposes the input data based on FWDM.

7.4.2 Uniform Template-based Accelerator for 2D and 3D CNNs. Shen et al. [54] designed an accelerator based on templates for both 2D and 3D CNNs which also integrates an analytical model for DSE. They improve on their previous work [53] by focusing on a deep-pipelined architecture with the goals of avoiding large off-chip memory transfers, under-utilization of FPGA resources, and at the same time improving throughput. Compared to 3D-VNPU [9] which counts on activation and filter decomposition, this work [54] applied the Winograd algorithm to reduce the computational complexity of the convolutions and discovered that the 1D Winograd transformation is a common operation of 2D and 3D convolutions. Similarly, the element-wise multiplication of a 2D input tile and a 2D filter tile can be used for element-wise multiplication on the 3D tiles by repeating the operation across the depth dimension. Furthermore, the accumulation of the elements in the transformed 2D output tiles is also a common operation. Based on these common operations, the authors propose an architecture which includes three types of templates: transformation, element-wise multiplication, and accumulation templates. The main component of the accelerator is the **Layer Computation Engine** (**LCE**). It is composed of input/weights/output buffers, PUs, and activation (ReLU) and pooling units. Each PU contains multiple PEs that integrate the computational templates.

The final design is a pipelined architecture with multiple LCEs. The layers assigned to the same LCE are executed in order, while the layers assigned to different LCEs are executed in parallel. Three more interesting techniques are applied: layer fusing, layer clustering and workload balancing. Layer fusing determines and groups layers with the same data reuse patterns, while layer clustering is an efficient way of mapping the convolutional layers onto the LCEs based on the tiling parameters. Furthermore, since the throughput of the accelerator depends on the slowest LCE in the pipeline, the authors implement a method to balance the workload across LCEs.

Figure 11 compares FPGA-based accelerators for C3D in terms of power efficiency and resource efficiency. Shen et al. [54] and 3D-VNPU [9] represent the best solutions with higher resource and power efficiency than the other accelerators. Fan et al. (2) [12] achieves competitive resource efficiency but lower power efficiency.

7.4.3 Uniform Architecture for Accelerating 2D and 3D CNNs. Compared to the uniform template-based accelerator [53], Liu et al. [42] map the convolutions to matrix multiplications. The disadvantage of the Winograd convolution is that for different convolutional kernel sizes, different transformation matrices need to be computed. Therefore, in order to design an accelerator suitable for multiple types of CNN architectures, the authors opt for implementing the convolution as matrix multiplications. This method increases the degree of data replication in the input features, but this is overcome by an efficient design of the matrix mapping module. A splitting strategy is

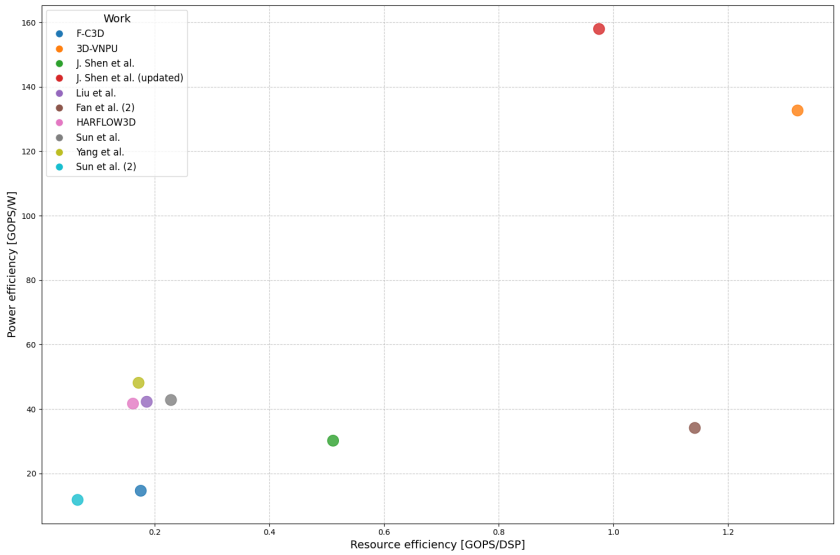


Fig. 11. Comparison between FPGA-based accelerators for C3D in terms of power efficiency and resource efficiency.

used to make sure that the input data fits in the on-chip memory. A convolutional layer which has many channels is split into multiple layers, each with subsets of channels, followed by a sum layer.

The architecture of the accelerator is based on the mapping module, a 2D MAC (Multiply and Accumulate) array used to compute the matrix multiplication, an ACC module for the accumulation of two convolutional layers, a BN module for batch normalization and a NL unit for the non-linear activation functions. Additionally, the architecture includes several buffers for features, weights and output and a controller which is used for instruction fetching.

7.5 Toolflows

7.5.1 HARFLOW3D. HARFLOW3D [63] is a latency oriented toolflow built on the fpgaConvNet [66] framework which maps CNNs onto FPGAs and which has been extended to support 3D CNNs. They provide results on multiple 3D CNNs with different architectures, among which we find C3D, R(2+1)D and X3D, all included in Table 8.

The toolflow includes a model parser, a method for performance and resource modeling, a scheduling algorithm for executing the 3D CNN on the generated MPE architecture, and a resource-aware optimization engine. The model parser maps the input 3D CNN from an **Open Neural Network Exchange** (**ONNX**) format to a **Synchronous Data-Flow Graph** (**SDFG**).

DSE is leveraged for performance and resource modeling. The focus is on minimizing latency, hence the performance modeling estimates the execution latency of each layer based on its parameters. Similarly, they use an analytical method to estimate the resource utilization of the design. The purpose of the optimization engine is to find the best execution schedule for the 3D CNN layers that minimizes the total latency.

The accelerator architecture proposed in this work consists of several building blocks corresponding to different types of layers in 3D CNNs which are controlled by a CPU through custom instructions, and crossbars for data routing between blocks and memory, or between blocks. The core building blocks support a wide range of layer sizes that can be tuned based on the target 3D CNN. The toolflow is in charge of detecting which computation blocks are used for a particular 3D

CNN, tuning them accordingly, constructing the execution schedule, and optimizing the design for latency minimization while obeying the resource limitations of the FPGA. This toolflow supports runtime configuration of the hardware which introduces a resource overhead. However, the authors state that this overhead is not significant.

7.5.2 fpgaHART. While HARFLOW3D [63] is a latency oriented toolflow which generates FPGA designs with an MPE architecture, fpgaHART [62] is a throughput oriented toolflow which generates FPGA designs with a streaming architecture for 3D CNNs.

Similar to HARFLOW3D, fpgaHART is derived from the fpgaConvNet [66] framework. The input 3D CNN is parsed to create a SDFG. Then, each node in the graph is mapped to its corresponding hardware building block. As discussed in Section 7.3.4 the main downside of streaming architectures is the high resource requirements. The authors use the same method of partitioning the design and reconfiguring the FPGA with every new partition as used in FMM-X3D.

One advantage of streaming architectures is that parallelism can be exploited per layer, but this is limited by the available hardware resources. The tool uses DSE to find the best solution for the input 3D CNN and the target FPGA. This involves optimization of the partitioning process and modeling resource utilization and throughput per individual layer as well as for the full design. This toolflow has been tested with the following 3D CNNs (all included in Table 8) : C3D, R(2+1)D-18, R(2+1)D-34 and X3D.

Both tools, HARFLOW3D [63] and fpgaHART [62], are in the early stages of maturity but they support a reasonable set of 3D CNN architectures and target devices. fpgaHART is open source, while HARFLOW3D has a backend which is closed source. The backend translates the layer configuration to a hardware IP which is private, hence the bitstream and the host code can be received only by requesting the files from the authors.

7.6 Design Optimization Techniques

7.6.1 Memory Management. Bringing input feature maps and parameters from off-chip memory to on-chip memory takes a large amount of time, and bandwidth is limited. Therefore, researchers use different strategies to minimize communication with off-chip memory.

Fan et al. [15] use an input cache strategy to exploit data reuse where they cache input frames until all kernels have been applied on the cached frame. However, depending on the size of the batch of frames, the cache may not be large enough to hold all of them. Thus, F-C3D considers two 3D blocking approaches: frame blocking and pixel blocking. Frame blocking maintains the size of the original frame but selects some of the frames from the input sequence, while pixel blocking divides each frame into squares of equal size. This architecture can be adapted for different FPGAs, and choosing the right 3D blocking approach depends on the size of the cache. Both methods have overhead, so for the layers where the input data cannot be entirely stored in cache, the approach with the least overhead is used. F-C3D was implemented on the Xilinx ZC706 SOC and the authors report that pixel blocking is more efficient in that case. Similarly to F-C3D [15], [36] proposes an input tiling method across all dimensions of the input data; however, this implies overlapped data and the same data may be read multiple times. In contrast, Tian et al. [60] explore memory access optimization methods and propose a data tiling method for contiguous off-chip memory access which does not overlap data. Their tiling method is applied over dimensions C and M where C is the number of channels and M is the number of filters. Xiang et al. [70] also apply tiling to the output channel and show that this method does not involve any overlapped data and thus ensures continuous off-chip memory access.

In a following work focused on **Block Floating Point** (**BFP**) representation, Fan et al. [14] find that a Filter/Kernel-Channel-Frame data access pattern improves data locality and reduces

the number of data transfers between the on-chip and off-chip memory. They also use a blocking method that divides the input frame in blocks and then aligns the data based on their exponents (block alignment). These techniques improve the degree of parallelism and efficiently organize the data in each block for BFP. An interesting takeaway from this work is that data alignment across the whole frame results in larger precision loss than data alignment in a single block. The choice of block size is also important as a small block size improves the accuracy but decreases the performance, and a large block size can significantly reduce the on-chip storage demand but at the cost of accuracy.

The method used in F-E3D [13], called online blocking, reduces the memory demand by computing only the depth-wise and point-wise convolutions on blocks of the input data. On-chip memory can hold the results of the temporal convolution because it has a lower number of output channels, but the intermediate data produced by the depth-wise and the point-wise convolutions do not fit on chip. After the temporal convolution, partial computations on blocks of data are executed and then sent to the off-chip memory which results in a lot of communication with the off-chip memory.

The HARFLOW3D toolflow [63] uses tiling of the feature maps; the optimum dimension of the tile depends on the size of the on-chip memory and is determined during the DSE phase of generating a custom design for the input 3D CNN.

In the case of streaming architectures, such as fpgaHART [62] and FMM-X3D [61], their advantage is that communication with off-chip memory can be limited to feeding only the input to the design and then getting the output. In a streaming design, the next computation block (corresponding to a layer in the network) starts computing as soon as the previous one finishes. However, there are cases where parameters still do not fit on chip and will have to be read from the off-chip memory ([11] is an example of a 2D CNN streaming hardware architecture with this issue). As discussed in Section 7.3.4, these works [61, 62] split the design into multiple partitions to overcome the problem of high resource requirement of streaming architectures. Hence, they manage to fit the parameters on chip and they reconfigure the FPGA with each partition. For each partition, they get the input stream from off-chip memory and then send the output off chip as well.

7.6.2 Parallelism. A common technique for exploiting parallelism is optimizing the loop of the convolution operation. Loop unrolling [13, 70, 74] increases parallelism and facilitates data reuse. Loop interchange is explored in [70] to maximize the computing efficiency of the PE arrays for both the 1D and 2D computing modules. Loop unrolling and loop interchange are also part of the design strategy of F-E3D [13] to exploit kernel reuse. Kernel reuse improves the hardware efficiency by reusing the computational kernel of the 3D depth-wise convolution to compute the temporal and point-wise convolutions.

Tian et al. [60] explore different strategies of feeding data to the accelerator by changing the loop order and leveraging feature and kernel parallelism. They observe that each 3D CNN layer may require a different loop ordering strategy and choosing one for all layers is not optimal. The authors also developed a software framework which can be used to find the best loop ordering strategy and tiling parameters for each layer of the 3D CNN.

Fan et al. [15] implement two types of parallelism: pixel and kernel parallelism. In kernel parallelism, multiple different 3D kernels (different coefficients) receive the same pixel, while in pixel parallelism, multiple 3D kernels with the same coefficients receive different pixels from the matrix buffer. The authors argue that kernel parallelism has simpler control logic and less overhead, hence they chose it for the F-C3D accelerator.

[36] presents an analysis of parallelism modes based on the way the tiled data is fetched (such as MIMO – Multiple Input Multiple Output or SIMO – Single Input Multiple Output) and conclude that different modes can be used for different stages in the network based on the resulting on-chip

memory requirements when using a certain mode for a particular layer. They present a comparison of each mode in terms of off-chip memory accesses versus BRAM utilization.

Shen et al. [54] parallelize the executions of PEs in each PU by splitting the input feature buffers into Ti blocks. Then, Ti input feature blocks corresponding to Ti PEs are read in parallel. They also partition the weight buffer to increase parallelism among PUs.

HARFLOW3D [63] uses coarse-grain folding and fine-grained folding transformations to leverage and adjust levels of parallelism. Through coarse-grain folding the number of computing instances for each layer can be adjusted; this represents parallelism over the channel dimensions of the input feature map where the maximum number of instances is the number of channels. Fine-grain folding enables parallelism at a lower level by adjusting the number of multipliers used and the depth of the adder trees when computing the 3D convolution. Parallelism is constrained by the number of available resources. The authors deploy the same strategy in fpgaHART [62].

7.6.3 Data Representation. There are two works which accelerate 3D CNNs using BFP representation [12, 14]. BFP aims at reducing the data bitwidth which in turn reduces the memory and DSP requirements. Generally, in the case of quantization, retraining is required to ensure the classification accuracy does not drop under a certain limit. BFP uses a shared exponent block instead of an individual exponent for each data as it is often the case that a block of data has the same dynamic range. Thus, the processing can use fixed point components which emulate floating point arithmetic and, because the computations are still based on floating point arithmetic, accuracy is maintained and retraining is not necessary. The processor is in charge of the blocking alignment. This is done right before sending the data to the on-chip memory where it is temporarily stored in a buffer. The convolution results are then realigned on the processor. The operations performed on the processor can be executed in parallel with the computations on the FPGA. Fan et al. [12] expand on their previous work [14] and propose the Static Block Floating Point Representation. This method uses fixed share exponents during inference and this eliminates the exponent calculation together with the frequent FP-BFP and BFP-FP conversions required in their first implementation. By removing these calculations, the computational and memory resources are used more efficiently.

7.6.4 Reducing Computational Complexity through Winograd Convolution. Shen et al. [54] compare the number of multiplications and additions needed with a Winograd 3D convolution to a normal 3D convolution implemented as matrix multiplication. The Winograd method reduces the number of multiplications but increases the number of additions. The example in [54] shows that for an ordinary 3D convolution with 3x3x3 kernels (common kernel size among the state-of-the-art 3D CNNs), the Winograd algorithm reduces the number of multiplications from 216 to 64 and increases the number of additions from 208 to 419. Since multiplications are more expensive, the computational complexity is reduced. Another advantage of Winograd is that it does not need to replicate data, as happens in the case of mapping the convolution to matrix multiplication.

While Shen et al. [54] consider 3D Convolutions with a stride of 1 (the step size by which the kernel moves over the input), Yepez et al. [75] apply the Winograd to 3D convolutions with a stride of 2 (3x3x3 kernels). This is important as there are 3D CNNs such as I3D and 3D ResNet which use a stride of 2. A similar conclusion is reached: the number of multiplication is reduced (from 216 to 125) and the number of additions is increased (from 216 to 419).

The Winograd-based FWDM technique in 3D-VNPU [9] also supports different convolution dimensions, kernel sizes and strides (1 and 2) which makes the accelerator highly scalable. Evaluation results show that FWDM reduces computational complexity by a factor of 3.2 for C3D, 2.9 for 3D ConvNet, and 2.6 for 3D ResNet-18. For instance, 3D ResNet-18 has four stride-2 layers which account for 44% of the total computation. Compared to convolution, FWDM reduces approximately 60 to 70% of the multiplication operations for 3D CNNs.

Table 10. Hardware-aware Algorithmic Optimizations for 3D CNNs

Work	Method	3D CNN	FPGA Implementation	Results
Sun et al. [58]	Blockwise pruning of weights	R(2+1)D	Yes (ZCU102)	2.6x speed-up compared to the unpruned version
Wang et al. [68]	Filter/Kernel pruning	C3D P3D ResNet	No	50.53% FLOPS reduction with 0.93% accuracy drop for C3D on Sports1M dataset
RANP [73]	Neuron pruning	I3D	No	49.9% FLOPS reduction 35.3% memory reduction
Zhang et al. [79]	Regularization-based weight group pruning	3D ResNet-18 C3D	No	2x theoretical speed-up with 0.41% accuracy loss for 3D ResNet-18 and 3.28% accuracy loss for C3D
Lee et al. [40]	Fixed Point Quantization	3D RestNet-10 3D RestNet-18 3D RestNet-34	No	up to 71% storage reduction
ATFR [16]	Learning the similarity of temporal features	R(2+1)D I3D, X3D 3D ResNet	No	33% to 53% GFLOPS reduction
Sun et al. (2) [57]	Kernel group row-column weight pruning Quantization	C3D R(2+1)D	Yes (ZCU102)	4.12x speed-up for C3D and 3.85x for R(2+1)D compared to baseline

8 Hardware-aware Algorithmic Optimizations for 3D CNNs

In this section, we focus on algorithmic optimizations that have been used for 3D CNNs where hardware implementations are taken into consideration. Many of these techniques have also been used for 2D CNNs; for more information see [20, 51]. Compression techniques such as pruning and quantization need to take into account the additional temporal dimension in 3D CNNs and their increased complexity. The goal of these optimizations is to accelerate both training and inference and to facilitate the hardware implementation of these neural networks. Table 10 summarizes the work presented in this section, the methods and 3D CNNs used, and the achieved results.

8.1 Pruning

Pruning reduces the computational complexity and the memory requirements of various types of neural networks by removing those components that have low impact on performance. In the context of 3D CNNs, pruning can significantly compress the network, speed up training and inference, and reduce the resource utilization on FPGAs. However, a large compression rate usually comes at the cost of lower accuracy.

There are multiple types of pruning in CNNs:

(1) Coarse-grained
  (a) Layer pruning - removing entire layers of the network. This changes the depth of the network and can lead to shallow networks and a significant drop in accuracy. However, C3D, a shallow network compared to deeper networks such as P3D ResNet, achieves a better accuracy than P3D ResNet on the UCF101 dataset.
  (b) Channel/Feature map pruning - if feature maps are pruned, kernels in the following layers are also removed.
  (c) Kernel pruning - removing entire kernels or individual kernels from a set.
(2) Fine-grained
  (a) Structured pruning – removing rows, columns, or blocks of weights.
  (b) Unstructured pruning – removing individual connections or weights.

The selection of components of the CNN that can be pruned is usually done through one of two main methods: importance-based methods and regularization-based methods. The goal is to achieve a compressed model and an improvement in latency with a minimum drop in accuracy.

Zhang et al. [79] use a regularization method that is a type of structured pruning. They compare two methods: **same pruning ratio** (**SPR**) and **different pruning ratio** (**DPR**). SPR uses the same pruning ratio for all convolutional layers, while DPR computes a different pruning ratio for each convolution layer based on redundancy and computational cost. Only the weights in the convolutional layers are compressed; the other layers are left as is. They test their approach on two 3D CNNs, C3D and 3D ResNet-18, on the UCF101 dataset and they obtain a 2× theoretical speed-up with 3.28% and 0.41% accuracy loss for C3D and 3D ResNet-18, respectively.

Wang et al. [68] focus on pruning convolutional kernels of 3D CNNs which are uninformative and too similar to other kernels. They evaluate their method on C3D and P3D ResNet, both trained on Sports-1M. The **Determinantal Point Processes** (**DPP**) method is used to measure the similarity between kernels and to find the least important and uninformative ones. Their method achieved a 50.53% FLOPS reduction on C3D with a 0.93% drop in top-1 accuracy and a 49.79% FLOPS reduction on P3D ResNet (199 layers) with a 0.81% drop in top-1 accuracy.

Xu et al. [73] introduce **Resource Aware Neuron Pruning** (**RANP**) and they use this method to prune the 3D CNNs at initialization. The neurons are pruned based on their importance given by the resource consumption of each neuron. Pruning at initialization is useful for reducing computational complexity and training time. For I3D, they obtain a 49.9% FLOPS and 35.3% memory reduction with a 2% accuracy increase. The experiments are done on GPUs and the authors explain that the accuracy increase is due to an enlarged batch size used on each GPU.

Sun et al. [58] apply blockwise weight pruning for FPGA acceleration of R(2+1)D. The weight tensor is divided in blocks of the same size as the FPGA buffer used to store the weights. The weights of the pruned blocks are set to 0 and the loss function is minimized using the **Alternating Direction Method of Multipliers** (**ADMM**). Masking and retraining was needed in order to minimize the accuracy drop. Masking prevents the zero weights from being updated during retraining. The pruned model implemented on the Xilinx ZCU102 FPGA achieves a 2.6X latency speed-up compared to the unpruned version, while the accuracy drop is less than 1%. In a following work, the authors [57] propose a kernel group sparsity method to achieve a high pruning ratio with a small loss in accuracy. In this case, the weight tensor is split into kernel groups of equal size and pruning is applied either to rows, columns, or both. This sparsity technique is called balanced because all kernel groups in a weight tensor have the same number of pruned rows and columns facilitating computation parallelism. With the kernel group sparsity and 8 bit quantization, a latency speed-up of 4.12x for C3D and 3.85x for R(2+1)D is achieved compared to the 16 bit baseline. Compared to [58] which applies blockwise weight pruning and which uses 16 bit precision, the latency speed up of [57] is 11.29x for C3D and 4.47x for R(2+1)D.

8.2 Quantization

For efficient 3D CNN hardware acceleration, quantization plays an important part. The ideal outcome of quantizing the network inputs, outputs or parameters would be to preserve the accuracy of the floating point model and decrease the memory requirement and the computation complexity of the hardware implementation. However, quantization usually comes with a tradeoff in accuracy. One challenge of design space exploration is extremely long training times.

Work [23, 69] on 2D CNNs for image classification tasks show that quantization can significantly speed-up the computation and reduce the storage needed by the models at a cost of less than 1% drop in accuracy. Furthermore, Blott et al. [4] show that among different computing platforms, FPGA implementations benefit the most from quantization in terms of latency and throughput.

The work in [40] explores fixed point quantization of 3D CNNs. They report results on 3D ResNet with 10, 18 and 34 layers. The most accurate model between the three is 3D ResNet-34 with a 32b floating-point accuracy of 84.11% on UCF101.

They perform an extensive search of the optimal quantization level for the kernels and features of each model, starting from the 32b floating-point model. The lowest quantization levels for features that don't allow an accuracy drop of more than 1% are 15 bit for all 3 networks, ResNet-10, ResNet-18 and ResNet-34. In the case of kernel quantization, they achieved 8 bit for ResNet-10, 10 bit for ResNet-18 and 9 bit for ResNet-34. Their implementations of the quantized models, which are designed for 65 nm CMOS technology based ASICs and not FPGAs, show promising reduction in storage size and required energy per action recognition of about 70% each.

In Section 7, we present other works that also use quantization. For example, Li et al. [41] implement a binarized 3D CNN and they also design a tailored accelerator. Fan et al. [14] use a block floating point representation with sign of 1 bit, a 15b mantissa and an 8 bit exponent. 3D-VNPU [9], Tian et al. [60] and Khan et al. [36] quantize the target 3D CNNs to 8 bits.

8.3 Adaptive Temporal Feature Resolution (ATFR)

The goal of this work [16] is to reduce the training and inference time of 3D CNNs. The temporal resolution, i.e., the number of frames of the input video, is generally fixed. On one hand, a high temporal resolution translates to a high computation time for both training and inference and increases the difficulty in data fetching and movement inside the accelerator. On the other hand, a low temporal resolution might not provide the network all the information needed for generating an accurate classification. The researchers propose a dynamic method of changing the temporal resolution within the neural network in order to get rid of redundant information and lower the computation time. They designed a **Similarity Guided Sampling** (**SGS**) module that finds similar feature maps and combines them into one single output feature map. The SGS module was implemented as a PyTorch layer and it can be easily added to existing 3D CNNs. This approach was tested on R(2+1)D, I3D, X3D, and a modified 3D ResNet, and on six different datasets. They obtained a reduction in the computation cost of 33% to 53% of GFLOPS while maintaining accuracy.

9 Conclusions and Future Directions

3D CNNs outperform 2D CNNs on the action recognition task by inherently handling temporal information and by taking advantage of multiple strategies such as pre-training on image and action recognition datasets, or using an additional optical flow input stream. 2D CNNs with two stream input, image and optical flow, achieve decent accuracy, the closest to 3D CNN's performance being a 2D CNN combined with a LSTM architecture which keeps track of the order of the frames. This is seen on the Sports-1M dataset; however, on the UCF101 and Kinetics dataset 3D CNNs achieve higher accuracy.

The price to pay for higher accuracy is defined mainly by the computational complexity and the storage requirements of 3D CNNs. These challenges become obvious especially when targeting implementation and acceleration of the networks on resource-limited hardware platforms, such as FPGAs. In this context, it is challenging to fit all the data needed for computation on-chip; therefore research is strongly focused on network compression and memory management. Data tiling and loop processing (unrolling, reordering) are among the most adopted techniques to deal with the large amounts of data and to increase parallelism. A few of these works provide an extensive analysis of the impact of their choices on the performance of the design. **Design Space Exploration** (**DSE**) methods remain crucial for discovering the best performance-resource utilization tradeoffs and for application-specific optimization. More extensive analysis would benefit the community.

Regarding the reviewed FPGA-based 3D CNN accelerators, in general, their performance exceeds that of CPUs and is lower than that of GPUs, but they are more power efficient than GPUs. We believe that these results could potentially be boosted by more aggressive hardware-aware optimization at the algorithmic level through quantization and pruning. Quantization and pruning are two

efficient methods of network compression. Pruning has been more widely explored compared to quantization. According to this work [40], the lowest bitwidth that does not result in more than 1% accuracy drop is 8 bit for weight quantization of 3D ResNet-10. 3D CNN quantization represents a research gap which, if explored, might lead to significant improvements in network compression and more efficient hardware accelerators. The research done on quantized 2D CNNs serves as inspiration; compelling results have been obtained even for binarized networks. Lower levels of quantization (< 8 bit), binarized 3D CNNs (with evaluation on larger datasets than KTH [52]), and layer-wise quantization where different bit widths can be used for each layer are some ideas worth studying. Furthermore, there is a need for a design space exploration tool that could help simplify the search for the best tradeoff between network compression ratio and accuracy drop.

We have also presented specialized designs, unified accelerators for 2D and 3D CNNs, and toolflows. All specialized designs reviewed in this survey are tailored for a different 3D CNN; therefore, they cannot be directly compared. However, since specialized designs are focused on a single architecture, it is expected that they have a higher performance and lower power consumption than more generic designs. The advantage of toolflows, compared to specialized designs, is that they can support a wider range of 3D CNN architectures and the designs can be adapted for different platforms. They also include methods of DSE which can help adapt the accelerator's design based on the neural network's needs, leading to better performance compared to generic designs. These are important characteristics especially as 3D CNNs tend to evolve fast. On the downside, for new 3D CNN architectures with different types of layers, the toolflows may need additional support, and efficient DSE may require the user to have some expertise with the tool. In terms of the unified accelerators for 2D and 3D CNNs, we noticed that for C3D, two of them [9, 54] have the best power and resource efficiency. Their common point is the Winograd transformation and we think this boosted their performance, along with other design techniques. A future direction could be integrating the Winograd convolution in the toolflows, to take advantage of both the reduced computational complexity and the flexibility of the toolflow. Another feature which is worth exploring more is the ATFR module which shows promising results in reducing the computation complexity. Integrating ATFR in FPGA-based designs for 3D CNNs could represent another path forward to more efficient accelerators.

Our survey shows that FPGAs are well suited for processing 3D CNNs where improved latency over CPUs is needed and power consumption can be reduced compared to GPU implementations. Recently new architectures have been introduced that add vector-based processing alongside FPGA fabric [19]. There are opportunities to explore tradeoffs among all of these different architectures in the future.

Acknowledgments

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of NSF.

References

[1] Sami Abu-El-Haija, Nisarg Kothari, Joonseok Lee, Paul Natsev, George Toderici, Balakrishnan Varadarajan, and Sudheendra Vijayanarasimhan. 2016. YouTube-8M: A large-scale video classification benchmark. Retrieved from https://arxiv.org/abs/1609.08675

[2] Muneer Al-Hammadi, Ghulam Muhammad, Wadood Abdul, Mansour Alsulaiman, and M. Shamim Hossain. 2020. Hand gesture recognition using 3D-CNN model. *IEEE Consumer Electronics Magazine* 9, 1 (2020), 95–101. DOI: https://doi.org/10.1109/MCE.2019.2941464

[3] Gabriel Bender, Hanxiao Liu, Bo Chen, Grace Chu, Shuyang Cheng, Pieter-Jan Kindermans, and Quoc V. Le. 2020. Can weight sharing outperform random architecture search? An investigation with tunas. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14323–14332.

[4] Michaela Blott, Nicholas J. Fraser, Giulio Gambardella, Lisa Halder, Johannes Kath, Zachary Neveu, Yaman Umuroglu, Alina Vasilciuc, Miriam Leeser, and Linda Doyle. 2021. Evaluation of optimized CNNs on heterogeneous accelerators using a novel benchmarking approach. *IEEE Trans. Comput.* 70, 10 (2021), 1654–1669. DOI: https://doi.org/10.1109/TC.2020.3022318

[5] Joao Carreira, Eric Noland, Andras Banki-Horvath, Chloe Hillier, and Andrew Zisserman. 2018. A short note about Kinetics-600. Retrieved from https://arxiv.org/abs/1808.01340

[6] Joao Carreira, Eric Noland, Chloe Hillier, and Andrew Zisserman. 2022. A short note on the Kinetics-700 human action dataset. Retrieved from https://arxiv.org/abs/1907.06987

[7] Joao Carreira and Andrew Zisserman. 2017. Quo vadis, action recognition? A new model and the kinetics dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Honolulu, HI, USA, 6299–6308.

[8] Shaoxiang Chen, Ting Yao, and Yu-Gang Jiang. 2019. Deep learning for video captioning: A review. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, Macao, China, 6283–6290. DOI: https://doi.org/10.24963/ijcai.2019/877

[9] Huipeng Deng, Jian Wang, Huafeng Ye, Shanlin Xiao, Xiangyu Meng, and Zhiyi Yu. 2021. 3D-VNPU: A flexible accelerator for 2D/3D CNNs on FPGA. In *Proceedings of the 2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, IEEE, Orlando, FL, USA, 181–185.

[10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *Proceedings of the 2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Miami, FL, USA, 248–255. DOI: https://doi.org/10.1109/CVPR.2009.5206848

[11] Dana Diaconu, Yanyue Xie, Mehmet Gungor, Suranga Handagala, Xue Lin, and Miriam Leeser. 2023. Machine learning across network-connected FPGAs. In *Proceedings of the 2023 IEEE High Performance Extreme Computing Conference* . IEEE, Boston, USA, 1–7. DOI: https://doi.org/10.1109/HPEC58863.2023.10363454

[12] Hongxiang Fan, Shuanglong Liu, Zhiqiang Que, Xinyu Niu, and Wayne Luk. 2023. High-performance acceleration of 2-D and 3-D CNNs on FPGAs using static block floating point. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2023), 4473–4487. DOI: https://doi.org/10.1109/TNNLS.2021.3116302

[13] Hongxiang Fan, Cheng Luo, Chenglong Zeng, Martin Ferianc, Zhiqiang Que, Shuanglong Liu, Xinyu Niu, and Wayne Luk. 2019. F-E3D: FPGA-based acceleration of an efficient 3D convolutional neural network for human action recognition. In *Proceedings of the 2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors*. IEEE, IEEE, New York, NY, USA, 1–8.

[14] Hongxiang Fan, Ho-Cheung Ng, Shuanglong Liu, Zhiqiang Que, Xinyu Niu, and Wayne Luk. 2018. Reconfigurable acceleration of 3D-CNNs for human action recognition with block floating-point representation. In *Proceedings of the 2018 28th International Conference on Field Programmable Logic and Applications*. IEEE, Dublin, Ireland, 287–2877. DOI: https://doi.org/10.1109/FPL.2018.00056

[15] Hongxiang Fan, Xinyu Niu, Qiang Liu, and Wayne Luk. 2017. F-c3d: Fpga-based 3-dimensional convolutional neural network. In *Proceedings of the 2017 27th International Conference on Field Programmable Logic and Applications*. IEEE, IEEE, Ghent, Belgium, 1–4.

[16] Mohsen Fayyaz, Emad Bahrami, Ali Diba, Mehdi Noroozi, Ehsan Adeli, Luc Van Gool, and Juergen Gall. 2021. 3D CNNs with adaptive temporal feature resolutions. In *Proceedings of the 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Nashville, TN, USA, 4729–4738. DOI: https://doi.org/10.1109/CVPR46437.2021.00470

[17] Christoph Feichtenhofer. 2020. X3d: Expanding architectures for efficient video recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Seattle, WA, USA, 203–213.

[18] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. 2019. Slowfast networks for video recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 6202–6211.

[19] Brian Gaide, Dinesh Gaitonde, Chirag Ravishankar, and Trevor Bauer. 2019. Xilinx adaptive compute acceleration platform: Versal architecture. In *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. ACM, USA, 84–93.

[20] Deepak Ghimire, Dayoung Kil, and Seong-heum Kim. 2022. A survey on efficient convolutional neural networks and hardware acceleration. *Electronics* 11, 6 (2022), 945.

[21] Lena Gorelick, Moshe Blank, Eli Shechtman, Michal Irani, and Ronen Basri. 2007. Actions as space-time shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 29, 12 (2007), 2247–2253. DOI: https://doi.org/10.1109/TPAMI.2007.70711

[22] Raghav Goyal, Samira Ebrahimi Kahou, Vincent Michalski, Joanna Materzynska, Susanne Westphal, Heuna Kim, Valentin Haenel, Ingo Fruend, Peter Yianilos, Moritz Mueller-Freitag, and others. 2017. The "something something" video database for learning and evaluating visual common sense. In *Proceedings of the IEEE International Conference on Computer Vision*. 5842–5850.

[23] Rishabh Goyal, Joaquin Vanschoren, Victor van Acht, and Stephan Nijssen. 2021. Fixed-point quantization of convolutional neural networks for quantized inference on embedded platforms. Retrieved from https://arxiv.org/abs/2102.02147

[24] Kensho Hara, Hirokatsu Kataoka, and Yutaka Satoh. 2017. Learning spatio-temporal features with 3d residual networks for action recognition. In *Proceedings of the IEEE International Conference on Computer Vision Workshops*. IEEE, Venice, Italy, 3154–3160.

[25] Kensho Hara, Hirokatsu Kataoka, and Yutaka Satoh. 2018. Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet?. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Salt Lake City, UT, USA, 6546–6555.

[26] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Las Vegas, NV, USA, 770–778.

[27] Fabian Caba Heilbron, Victor Escorcia, Bernard Ghanem, and Juan Carlos Niebles. 2015. ActivityNet: A large-scale video benchmark for human activity understanding. In *Proceedings of the 2015 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Boston, MA, USA, 961–970. DOI: https://doi.org/10.1109/CVPR.2015.7298698

[28] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, and others. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 1314–1324.

[29] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient convolutional neural networks for mobile vision applications. Retrieved from https://arxiv.org/abs/1704.04861

[30] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50× fewer parameters and <0.5MB model size. Retrieved from https://arxiv.org/abs/1602.07360

[31] Shuiwang Ji, Wei Xu, Ming Yang, and Kai Yu. 2013. 3D convolutional neural networks for human action recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35, 1 (2013), 221–231. DOI: https://doi.org/10.1109/TPAMI.2012.59

[32] Kaavya Kanagaraj and GG Lakshmi Priya. 2021. A new 3D convolutional neural network (3D-CNN) framework for multimedia event detection. *Signal, Image and Video Processing* 15, 4 (2021), 779–787.

[33] Andrej Karpathy, George Toderici, Sanketh Shetty, Thomas Leung, Rahul Sukthankar, and Li Fei-Fei. 2014. Large-scale video classification with convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Columbus, OH, USA, 1725–1732.

[34] Will Kay, Joao Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheendra Vijayanarasimhan, Fabio Viola, Tim Green, Trevor Back, Paul Natsev, Mustafa Suleyman, and Andrew Zisserman. 2017. The kinetics human action video dataset. Retrieved from https://arxiv.org/abs/1705.06950

[35] Fatima Hameed Khan, Muhammad Adeel Pasha, and Shahid Masud. 2024. Exploring memory access techniques for efficient FPGA based 3D CNN accelerator design. In *Proceedings of the 2024 IEEE 6th International Conference on AI Circuits and Systems*. IEEE, Abu Dhabi, United Arab Emirates, 218–222. DOI: https://doi.org/10.1109/AICAS59952.2024.10595963

[36] Fatima Hameed Khan, Muhammad Adeel Pasha, and Shahid Masud. 2023. Towards designing a hardware accelerator for 3D convolutional neural networks. *Computers and Electrical Engineering* 105 (2023), 108489.

[37] Dan Kondratyuk, Liangzhe Yuan, Yandong Li, Li Zhang, Mingxing Tan, Matthew Brown, and Boqing Gong. 2021. Movinets: Mobile video networks for efficient video recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16020–16030.

[38] Okan Kopuklu, Neslihan Kose, Ahmet Gunduz, and Gerhard Rigoll. 2019. Resource efficient 3d convolutional neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*. IEEE, Seoul, Korea (South), 0–0.

[39] H. Kuehne, H. Jhuang, E. Garrote, T. Poggio, and T. Serre. 2011. HMDB: A large video database for human motion recognition. In *Proceedings of the 2011 International Conference on Computer Vision*. IEEE, Barcelona, Spain, 2556–2563. DOI: https://doi.org/10.1109/ICCV.2011.6126543

[40] Hyunhoon Lee, Younghoon Byun, Seokha Hwang, Sunggu Lee, and Youngjoo Lee. 2018. Fixed-point quantization of 3D convolutional neural networks for energy-efficient action recognition. In *Proceedings of the 2018 International SoC Design Conference*. IEEE, Daegu, Korea (South), 129–130. DOI: https://doi.org/10.1109/ISOCC.2018.8649987

[41] Guoqing Li, Meng Zhang, Qianru Zhang, and Zhijian Lin. 2022. Efficient binary 3D convolutional neural network and hardware accelerator. *Journal of Real-Time Image Processing* 19, 1 (2022), 61–71. DOI: https://doi.org/10.1007/s11554-021-01161-4

[42] Zhiqiang Liu, Paul Chow, Jinwei Xu, Jingfei Jiang, Yong Dou, and Jie Zhou. 2019. A uniform architecture design for accelerating 2D and 3D CNNs on FPGAs. *Electronics* 8, 1 (2019), 65.

[43] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. Shufflenet v2: Practical guidelines for efficient cnn architecture design. In *Proceedings of the European conference on computer vision (ECCV)*. 116–131.

[44] S. Niyas, S. J. Pawan, M. Anand Kumar, and Jeny Rajan. 2022. Medical image segmentation with 3D convolutional neural networks: A survey. *Neurocomputing* 493 (2022), 397–413.

[45] Prachi Pardhi, Kiran Yadav, Siddhansh Shrivastav, Satya Prakash Sahu, and Deepak Kumar Dewangan. 2021. Vehicle motion prediction for autonomous navigation system using 3 dimensional convolutional neural network. In *Proceedings of the 2021 5th International Conference on Computing Methodologies and Communication*. IEEE, Erode, India, 1322–1329. DOI: https://doi.org/10.1109/ICCMC51019.2021.9418449

[46] Zhaofan Qiu, Ting Yao, and Tao Mei. 2017. Learning spatio-temporal representation with pseudo-3d residual networks. In *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, Venice, Italy, 5533–5541.

[47] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*. Springer, 525–542.

[48] Bin Ren, Mengyuan Liu, Runwei Ding, and Hong Liu. 2024. A survey on 3d skeleton-based action recognition using learning method. *Cyborg and Bionic Systems* 5 (2024), 0100.

[49] Mohammad Sabokrou, Mohsen Fayyaz, Mahmood Fathy, and Reinhard Klette. 2017. Deep-cascade: Cascading 3D deep neural networks for fast anomaly detection and localization in crowded scenes. *IEEE Transactions on Image Processing* 26, 4 (2017), 1992–2004. DOI: https://doi.org/10.1109/TIP.2017.2670780

[50] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 4510–4520.

[51] Arish Sateesan, Sharad Sinha, Smitha KG, and AP Vinod. 2021. A survey of algorithmic and hardware optimization techniques for vision convolutional neural networks on FPGAs. *Neural Processing Letters* 53, 3 (2021), 2331–2377.

[52] C. Schuldt, I. Laptev, and B. Caputo. 2004. Recognizing human actions: A local SVM approach. In *Proceedings of the 17th International Conference on Pattern Recognition*. IEEE, Cambridge, UK, 32–36 Vol.3. DOI: https://doi.org/10.1109/ICPR.2004.1334462

[53] Junzhong Shen, You Huang, Zelong Wang, Yuran Qiao, Mei Wen, and Chunyuan Zhang. 2018. Towards a uniform template-based architecture for accelerating 2D and 3D CNNs on FPGA. In *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays* (Monterey, CALIFORNIA, USA). Association for Computing Machinery, New York, NY, USA, 97–106. DOI: https://doi.org/10.1145/3174243.3174257

[54] Junzhong Shen, You Huang, Mei Wen, and Chunyuan Zhang. 2020. Toward an efficient deep pipelined template-based architecture for accelerating the entire 2-D and 3-D CNNs on FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 7 (2020), 1442–1455. DOI: https://doi.org/10.1109/TCAD.2019.2912894

[55] Satya P. Singh, Lipo Wang, Sukrit Gupta, Haveesh Goli, Parasuraman Padmanabhan, and Balázs Gulyás. 2020. 3D deep learning on medical images: A review. *Sensors* 20, 18 (2020), 5097.

[56] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. 2012. UCF101: A dataset of 101 human actions classes from videos in the wild. Retrieved from https://arxiv.org/abs/1212.0402

[57] Mengshu Sun, Kaidi Xu, Xue Lin, Yongli Hu, and Baocai Yin. 2024. Hardware-friendly 3-D CNN acceleration with balanced kernel group sparsity. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3027–3040. DOI: https://doi.org/10.1109/TCAD.2024.3390040

[58] Mengshu Sun, Pu Zhao, Mehmet Gungor, Massoud Pedram, Miriam Leeser, and Xue Lin. 2020. 3D CNN acceleration on FPGA using hardware-aware pruning. In *Proceedings of the 2020 57th ACM/IEEE Design Automation Conference* . IEEE, San Francisco, CA, USA, 1–6. DOI: https://doi.org/10.1109/DAC18072.2020.9218571

[59] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. 2015. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Boston, MA, USA, 1–9.

[60] Teng Tian, Xi Jin, Letian Zhao, Xiaotian Wang, Jie Wang, and Wei Wu. 2020. Exploration of memory access optimization for FPGA-based 3D CNN accelerator. In *Proceedings of the 2020 Design, Automation and Test in Europe Conference and Exhibition*. IEEE, IEEE, Grenoble, France, 1650–1655.

[61] Petros Toupas, Christos-Savvas Bouganis, and Dimitrios Tzovaras. 2023. FMM-X3D: FPGA-based modeling and mapping of X3D for human action recognition. In *Proceedings of the 2023 IEEE 34th International Conference on Application-specific Systems, Architectures and Processors*. IEEE, Porto, Portugal, 119–126. DOI: https://doi.org/10.1109/asap57973.2023.00030

[62] Petros Toupas, Christos-Savvas Bouganis, and Dimitrios Tzovaras. 2023. fpgaHART: A toolflow for throughput-oriented acceleration of 3D CNNs for HAR onto FPGAs. In *Proceedings of the 2023 33rd International Conference on Field-Programmable Logic and Applications*. IEEE, Gothenburg, Sweden, 86–92. DOI: https://doi.org/10.1109/fpl60245.2023.00020

[63] Petros Toupas, Alexander Montgomerie-Corcoran, Christos-Savvas Bouganis, and Dimitrios Tzovaras. 2023. HARFLOW3D: A latency-oriented 3D-CNN accelerator toolflow for HAR on FPGA devices. In *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 144–154.

[64] Du Tran, Lubomir Bourdev, Rob Fergus, Lorenzo Torresani, and Manohar Paluri. 2015. Learning spatiotemporal features with 3d convolutional networks. In *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, Santiago, Chile, 4489–4497.

[65] Du Tran, Heng Wang, Lorenzo Torresani, Jamie Ray, Yann LeCun, and Manohar Paluri. 2018. A closer look at spatiotemporal convolutions for action recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Salt Lake City, UT, USA, 6450–6459.

[66] Stylianos I. Venieris and Christos-Savvas Bouganis. 2016. fpgaConvNet: A framework for mapping convolutional neural networks on FPGAs. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 40–47. Retrieved from https://arxiv.org/abs/1711.08740

[67] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. 2018. Non-local neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 7794–7803.

[68] Zhenzhen Wang, Weixiang Hong, Yap-Peng Tan, and Junsong Yuan. 2020. Pruning 3D filters for accelerating 3D convnets. *IEEE Transactions on Multimedia* 22, 8 (2020), 2126–2137. DOI: https://doi.org/10.1109/TMM.2019.2950523

[69] Jiaxiang Wu, Cong Leng, Yuhang Wang, Qinghao Hu, and Jian Cheng. 2016. Quantized convolutional neural networks for mobile devices. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Las Vegas, NV, USA, 4820–4828.

[70] Dehao Xiang, Chenyang Li, Wenjin Huang, and Yihua Huang. 2024. A novel FPGA accelerator of R(2+1)D. In *Proceedings of the 2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines*. IEEE, Orlando, FL, USA, 1–7. DOI: https://doi.org/10.1109/FCCM60383.2024.00031

[71] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. 2017. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Honolulu, HI, USA, 1492–1500.

[72] Saining Xie, Chen Sun, Jonathan Huang, Zhuowen Tu, and Kevin Murphy. 2018. Rethinking spatiotemporal feature learning: Speed-accuracy trade-offs in video classification. In *Proceedings of the Computer Vision – ECCV 2018*. Vittorio Ferrari, Martial Hebert, Cristian Sminchisescu, and Yair Weiss (Eds.), Springer International Publishing, Cham, 318–335.

[73] Zhiwei Xu, Thalaiyasingam Ajanthan, Vibhav Vineet, and Richard Hartley. 2020. RANP: Resource aware neuron pruning at initialization for 3D CNNs. In *Proceedings of the 2020 International Conference on 3D Vision*. IEEE, IEEE, Fukuoka, Japan, 1–10.

[74] Yuesong Yang, Wenjing He, Jian Hu, Sai Cheng, and Wanqiu Xu. 2023. FPGA-based design for accelerating 3D convolutional neural networks. *International Journal of Frontiers in Engineering Technology* 5, 3 (2023), 40–48.

[75] Juan Yepez and Seok-Bum Ko. 2020. Stride 2 1-D, 2-D, and 3-D winograd for convolutional neural networks. *IEEE Transactions on Very Large Scale Integration Systems* 28, 4 (2020), 853–863. DOI: https://doi.org/10.1109/TVLSI.2019.2961602

[76] Jongmin Yu, Kin Choong Yow, and Moongu Jeon. 2018. Joint representation learning of appearance and motion for abnormal event detection. *Machine Vision and Applications* 29, 7 (2018), 1157–1170.

[77] Joe Yue-Hei Ng, Matthew Hausknecht, Sudheendra Vijayanarasimhan, Oriol Vinyals, Rajat Monga, and George Toderici. 2015. Beyond short snippets: Deep networks for video classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Boston, MA, USA, 4694–4702.

[78] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. ShuffleNet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 6848–6856.

[79] Yuxin Zhang, Huan Wang, Yang Luo, Lu Yu, Haoji Hu, Hangguan Shan, and Tony Q. S. Quek. 2019. Three-dimensional convolutional neural network pruning with regularization-based method. In *Proceedings of the 2019 IEEE International Conference on Image Processing*. IEEE, Taipei, Taiwan, 4270–4274. DOI: https://doi.org/10.1109/ICIP.2019.8803541

Received 25 October 2023; revised 14 October 2025; accepted 7 November 2025